



Class-based Obfuscation with Multi-objective Evolutionary Algorithms on Source Code

Guan-Yan Yang¹ · Tsung-Han Liu¹ · Farn Wang¹ · Kuo-Hui Yeh^{2,3}

Received: 4 October 2025 / Accepted: 24 June 2026
© The Author(s) 2026

Abstract

With the proliferation of software, protecting intellectual property from reverse engineering has become a paramount concern. Malicious actors who successfully decipher program logic can replicate applications, inject malicious code, and compromise user security. Code obfuscation serves as a critical defense mechanism by transforming a program's structure to impede analysis while preserving its original functionality. This process inherently involves a trade-off between maximizing the confusion introduced (potency) and minimizing performance degradation (cost), which defines a multi-objective optimization problem. In this study, we propose MOCCO (Multi-Objective Class-based Code Obfuscation), a novel framework that leverages Multi-Objective Evolutionary Algorithms (MOEAs) to automate and optimize the source code obfuscation process. Our primary contribution is the introduction and integration of class-based obfuscation techniques, which manipulate object-oriented constructs—an aspect often neglected by existing tools. We model obfuscation as a search for optimal solutions on a Pareto front, balancing metrics such as cyclomatic complexity, nesting level, code similarity, and execution time. We implement and compare three distinct MOEAs within our framework, including a global-update adaptation of MOEA/D. Our experimental evaluation, conducted on a set of benchmark programs, demonstrates that our approach, particularly when incorporating class-based techniques, generates code that scores substantially higher on structural-complexity (potency) proxies, with comparable or, in most cases, substantially lower performance overhead, relative to code produced by existing open-source obfuscators under comparable conditions.

Keywords Code Obfuscation · Software Protection · Multi-Objective Evolutionary Algorithms · Search-Based Software Engineering · Abstract Syntax Tree · Software Security

1 Introduction

The rapid growth of software applications has amplified challenges related to intellectual property protection and security [1–4]. Malicious actors frequently employ reverse engineering to comprehend program logic, allowing them to create counterfeit programs, inject malicious code, and expose users to significant security risks. The need for automated systems to detect vulnerabilities and secure software assets is therefore more pressing than ever [5–7].

Code obfuscation is a critical technique for this purpose, transforming a program's code to make it difficult to understand and analyze without altering its functionality [8]. By increasing a program's structural complexity, code obfuscation raises the cost and effort required for an attacker to comprehend its internal workings. While not providing absolute protection, it serves as a vital layer of defense.

Guan-Yan Yang and Tsung-Han Liu contributed equally to this work.

✉ Kuo-Hui Yeh
khyeh@nycu.edu.tw
Guan-Yan Yang
f11921091@ntu.edu.tw
Farn Wang
farn@ntu.edu.tw

- ¹ Department of Electrical Engineering, National Taiwan University, No. 1, Sec. 4, Roosevelt Rd., 106319 Taipei, Taiwan (R.O.C.)
- ² Institute of Artificial Intelligence Innovation, National Yang Ming Chiao Tung University, No. 1001, Daxue Rd., East Dist., 300093 Hsinchu City, Taiwan (R.O.C.)
- ³ Department of Information Management, National Dong Hwa University, No. 1, Sec. 2, Da Hsueh Rd., 974301 Shoufeng, Hualien, Taiwan (R.O.C.)

Obfuscation techniques are broadly categorized into layout, data, and control flow obfuscation [8]. These techniques, while effective, present a fundamental trade-off: increasing obfuscation potency often leads to performance overhead. This challenge of balancing competing goals makes code obfuscation an ideal candidate for multi-objective optimization.

In this work, we present MOCCO (Multi-Objective Class-based Code Obfuscation), a framework that utilizes Multi-Objective Evolutionary Algorithms (MOEAs) to navigate this complex solution space. Our framework operates on the Abstract Syntax Tree (AST) representation of source code. Because the optimization engine manipulates the AST as a generic tree and delegates all language-specific details to the parser, unparser, and transformation operators, the engine itself is, in principle, portable across languages; the AST grammar and the operators, however, are necessarily language-specific. We therefore implement and evaluate the framework on Python; broad cross-language applicability is a design goal we leave to future work (Section 6). We choose Python as our proof-of-concept target for the reasons given below. Python is often distributed as bytecode (`.pyc`) that decompilers can reverse-engineer back toward source; although individual tools track particular interpreter versions, the decompilability of CPython bytecode is well established [9, 10]. This vulnerability makes Python an especially relevant target for robust obfuscation techniques.

A key innovation of our work is to introduce new class-based operators and, to our knowledge, to be the first to drive class-based obfuscation with a multi-objective evolutionary search. Object-oriented obfuscation has been explored before (such as [11]) but has been largely neglected by modern tools and never cast as a Pareto-optimization problem. These techniques leverage object-oriented programming paradigms, such as inheritance and class composition, to introduce complexity in ways that traditional obfuscators often overlook. By integrating these novel techniques into the MOEA-driven optimization process, we aim to discover more potent and efficient obfuscation strategies.

Our research offers the following contributions:

- We propose MOCCO, a comprehensive, AST-based obfuscation framework that employs MOEAs to optimize multiple conflicting metrics, balancing obfuscation potency and performance cost.
- We introduce novel class-based obfuscation operators, expanding the repertoire of transformations available for securing object-oriented source code.
- We implement and rigorously compare three MOEAs within our framework—NSGA-II, MOEA/D, and a global-update variant (MOEA/D/ALT)—and find that decomposition-based search significantly outperforms

dominance-based search on the structural-complexity objectives at the higher length ratios, while broadening the update scope (MOEA/D/ALT) yields no systematic gain over standard MOEA/D.

- We conduct a thorough experimental evaluation, comparing our approach against established open-source obfuscators. Our results are analyzed based on equivalent program length inflation, providing a fair and practical assessment of obfuscation effectiveness.
- We release the framework, benchmark suite, test cases, and analysis scripts as a publicly archived artifact (see the Data Availability statement), enabling practitioners to use the tool and researchers to reproduce and extend our results.

The remainder of this paper is structured as follows: Section 2 reviews prior work. Section 3 details our proposed framework. Section 4 describes the experimental design. Section 5 presents and discusses the results. Section 6 discusses the threats to validity and limitations. Finally, Section 7 concludes the paper and outlines future research directions.

2 Related Works

This section reviews literature relevant to our study, covering encryption as a related security measure, existing obfuscation tools, multi-objective evolutionary algorithms, and prior work on applying MOEAs to code obfuscation. The field is continually evolving, with research focusing on new techniques, their detection, and ethical implications [12], as well as the impact of combined techniques [13].

2.1 Encryption

Encryption transforms data into an unreadable format accessible only to authorized parties. While distinct from obfuscation, it is sometimes used in conjunction with it. The primary drawback of relying solely on encryption for software protection is its fundamental vulnerability during execution. For the program to run, the encrypted code or data must be decrypted, which requires the decryption key to be present in the system's memory. A determined attacker can extract this key from memory at runtime, rendering the protection ineffective. This key management challenge limits the utility of pure encryption for protecting executable logic. Several studies have explored combining encryption with obfuscation. For example, [14] proposed PHPCrypton, which combines layout obfuscation with AES-256 encryption for PHP, and [15] presented a layout-based obfuscation method for C using hash functions and DES encryption.

2.2 Existing Obfuscation Tools

Numerous tools exist for code obfuscation, from open-source projects to commercial products. PyArmor [16] is a prominent example in the Python ecosystem, protecting scripts via obfuscation, function-level encryption, and a licensing system. However, its reliance on a runtime library to decrypt code imposes a performance penalty and introduces an external dependency. This overhead motivates obfuscation-only approaches, like ours, that seek to maximize confusion without runtime decryption. The effectiveness of tools like PyArmor has been evaluated in various contexts, including the obfuscation of malware [17–19].

2.3 Multi-objective Evolutionary Algorithms

Multi-Objective Evolutionary Algorithms (MOEAs) are population-based metaheuristics for problems with multiple, conflicting objectives [20]. Key categories include:

- **Domination-based MOEAs**, such as NSGA-II [21] and SPEA2 [22], use Pareto dominance to rank solutions and employ mechanisms like crowding distance to maintain diversity.
- **Indicator-based MOEAs**, like IBEA [23] and SMS-EMOA [24], incorporate performance metrics (indicators) directly into the selection process to guide the search.
- **Decomposition-based MOEAs**, notably MOEA/D [25], transform the multi-objective problem into a set of single-objective scalar subproblems, which are optimized simultaneously.

2.4 Obfuscation with Multi-Objective Evolutionary Algorithms

The application of MOEAs to code obfuscation is an active area of research. Many approaches leverage existing compiler frameworks. [26] proposed ShadObf, which used an evolutionary algorithm to select transformations from the PIPS source-to-source compiler for C code, and later extended the idea to JavaScript with JShadObf [27]. The LLVM compiler infrastructure is another common platform. [28] employed a cooperative co-evolutionary genetic algorithm to find optimal sequences of obfuscation passes from the Tigress and LLVM toolchains. [29] explored how native LLVM optimization passes could be repurposed for control flow obfuscation. [30] introduced an MOEA to balance complexity and performance using LLVM passes and later compared single- and multi-objective approaches [31].

Our work builds on these foundations, distinguishing itself by operating directly on the AST and introducing novel class-based transformations not typically found in compiler pass

libraries. Table 1 highlights the key differences between our approach and prior work.

3 Obfuscation Methodology

This section details our proposed obfuscation framework, MOCCO, depicted in Figure 1. The process begins by parsing the input source code into an Abstract Syntax Tree (AST). This AST serves as the foundational individual for initializing the MOEA's population. The population then undergoes an evolutionary process of mutation, evaluation, and selection. Finally, high-quality individuals (obfuscated ASTs) are selected from the Pareto front and unparsed back into source code.

3.1 Evolutionary Phases

Our framework follows a standard evolutionary algorithm structure. The specifics of Initialization and Selection depend on the chosen MOEA, as described in Section 3.2.

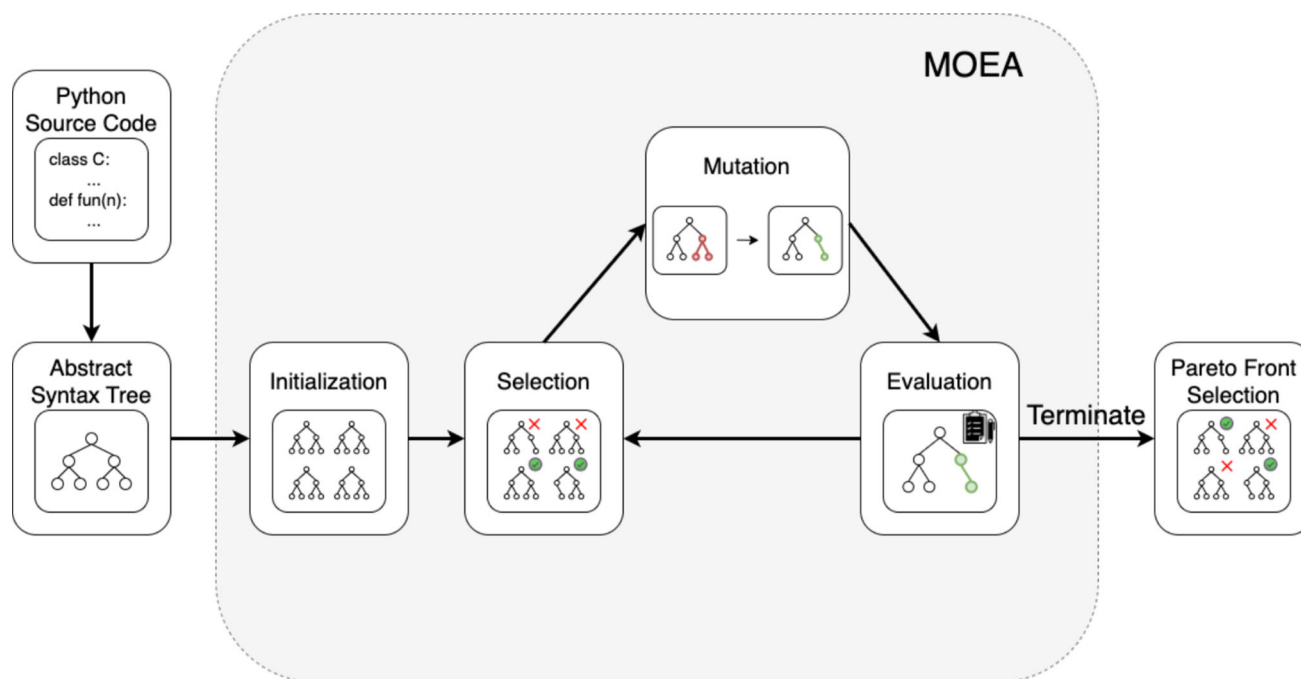
We deliberately omit a **Crossover** phase in our framework. Given the tree-based structure of our individuals (ASTs), designing a crossover operator that is both meaningful and consistently semantics-preserving is highly complex. Simple subtree swapping can easily violate syntactic or semantic rules of the programming language, resulting in invalid programs. Creating a robust AST crossover operator would require sophisticated validation and repair mechanisms, significantly increasing the computational cost and complexity of the framework. We concluded that the high implementation and runtime costs outweighed the potential, but not guaranteed, benefits for this study.

Crossover is only one of the two mechanisms by which evolutionary algorithms share information between candidate solutions; the second, *selection/replacement*, is preserved in full. The search therefore retains the cooperative behavior of population-based metaheuristics through the following mechanisms:

- **Elitist, comparison-based survival (NSGA-II)**. Offspring do not survive in isolation. The combined parent-and-offspring population is sorted into non-dominated fronts, and survival is decided by Pareto dominance and crowding distance computed *over the whole population*. A mutation is thus retained only if it is competitive against, and complementary to, the other individuals, so the trajectory of any one solution is continuously shaped by the rest of the population.
- **Neighborhood-based information sharing (MOEA/D)**. A new solution is generated from, and used to update, the solutions of an entire neighborhood of scalar subproblems defined by adjacent weight vectors. High-quality

Table 1 Comparison of MOEA-based Obfuscation Frameworks

Framework	Target	Granularity	Techniques	MOEA
ShadObf [26]	C	Source-to-Source	Traditional	NSGA-II
JShadObf [27]	JavaScript	Source-to-Source	Traditional	NSGA-II
LLVMObf [30]	C/C++	LLVM IR	Traditional	NSGA-II
Aragón-Jurado et al. [28]	C/C++	LLVM IR	Traditional	Co-evolutionary GA
Aragón-Jurado et al. [31]	C/C++	LLVM IR	Traditional	Co-evolutionary GA, NSGA-II
MOCCO (Our Work)	Python	AST (Source)	Traditional + Class-based	NSGA-II, MOEA/D, MOEA/D/ALT

**Fig. 1** Framework of the proposed class-based obfuscation using MOEAs

genetic material therefore propagates along the chain of overlapping neighborhoods, so subproblems cooperate to cover the Pareto front even though each mutation is local.

- **Global propagation (MOEA/D/ALT).** Our variant additionally allows a strong solution to update *all* subproblems, further amplifying cooperative dissemination (Section 3.2).

This places our algorithms in the well-studied family of mutation-only (crossover-free) MOEAs, which remain population-based because selection pressure couples the individuals. Consistent with this, the choice of selection/replacement scheme materially affects the outcome: under identical operators and budget, the decomposition-based algorithms (MOEA/D and MOEA/D/ALT) significantly outperform the dominance-based NSGA-II on the structural-potency objectives at the higher length ratios (Section 5, Table 6)—direct evidence that the population cooperates

through selection rather than evolving as a set of independent random walks.

The **Mutation** phase is where obfuscating transformations are applied. For each individual selected for mutation, exactly *one* transformation is applied per mutation event: the operator is drawn uniformly at random from the set of techniques listed below that are *applicable* to the current AST (e.g., Class Merging requires at least two classes, Function Inlining requires a callable target). Applying a single operator per event keeps the effect of each transformation observable to the selection step and lets potency accumulate incrementally over generations, while the per-generation re-application means an individual typically receives many transformations over a run. Operators that select a portion of a program element do so stochastically: for **Class Splitting**, the subset of members moved to the new (base or delegate) class is chosen uniformly at random, with the split point sampled so that neither side is empty; for **Function Outlining**, a contiguous, side-effect-consistent statement block is selected

at random; and for **Dead Code Insertion** and **Code Reordering**, the insertion point and the swapped pair of independent statements are likewise chosen at random. This randomized parameterization is intentional: it diversifies the offspring so that the population-level selection mechanism, rather than a fixed heuristic, decides which transformation magnitudes are retained. We group the operators below for clarity.

3.1.1 Class-based Techniques

- **Class Splitting (Delegate-based):** (Our proposal) A duplicate of an existing class is created. The methods in the original class are rewritten to delegate their calls to the corresponding methods in the new duplicate class.
- **Class Splitting (Inheritance-based):** A portion of a class's members is extracted into a new base class. The original class is then modified to inherit from this new superclass [11].
- **Class Merging:** Two existing classes are merged into a single new class that combines their functionalities [11].
- **Class Renaming (Polymorphic):** (Our proposal) Methods with different names across two or more classes are renamed to share the same name, creating apparent polymorphism and obscuring their distinct origins.

Rationale and impact on the obfuscation metrics. Traditional obfuscators operate almost exclusively at the statement and control-flow level, leaving the object-oriented *structure* of a program—its class hierarchy, member-to-class assignment, and method dispatch—essentially intact. This structure is precisely what a reverse engineer uses to recover the program's high-level design. Our class-based operators were chosen to attack this previously untouched dimension, and each is targeted at a specific way of degrading an analyst's mental model while moving the metrics we optimize:

- **Class Splitting** (both variants) increases the number of indirections an analyst must follow to reconstruct a single logical operation. Delegation and inheritance introduce additional call/dispatch edges into the control-flow graph, which raises *cyclomatic complexity*, and the redistribution and renaming of members substantially lowers token-level *similarity* to the original. Because the moved members execute the same code through one extra (cheap) dispatch, the effect on *running time* is negligible.
- **Class Merging** destroys the one-class-one-responsibility correspondence that analysts rely on, fusing unrelated members into a single namespace and lowering similarity, again with little control-flow cost.
- **Class Renaming (Polymorphic)** attacks identifier-based reasoning: by making semantically distinct methods share a name, it defeats the heuristic that “same name

implies same behavior” and lowers similarity without any runtime cost.

This rationale is borne out empirically. Our class-based operators improve the structural-potency metrics (cyclomatic complexity, similarity) with no statistically significant running-time penalty, but—consistent with the analysis above—they do *not* significantly affect nesting level, because they manipulate inter-class structure rather than intra-procedural nesting (Section 5, Table 5). Appendix 1 provides concrete, end-to-end illustrations: for example, the 1.5× Generate Parentheses listing shows a single recursive routine split across a base/derived class pair connected by delegation, and the 3× Invert Binary Tree listing shows a chain of delegating helper classes, both of which obscure the original single-class design while remaining functionally identical to the source program.

Behavior preservation. Because these operators are refactorings, preserving observable behavior is a correctness requirement, not merely a desirable property. We preserve it through two complementary mechanisms. First, *by construction*: every operator is a semantics-preserving refactoring in the classical sense [32]. Class Splitting relocates members but rewrites the original call sites to delegate (or inherit) so that every public method resolves to the same implementation; Class Merging performs the inverse, taking the union of disjoint members and rejecting the transformation when name collisions would shadow a member; Class Renaming is applied as a consistent rename, i.e., a definition and *all* of its references are updated together, and is only applied when the unified name does not clash within a scope. The traditional operators are likewise standard equivalence-preserving transformations (e.g., control-flow flattening reproduces the original execution order via the dispatcher, and dead-code insertion adds only provably unreachable or effect-free statements). Second, *by validation*: any candidate that fails to parse, fails to unparse, or raises an exception is discarded during evaluation, and—most importantly—we execute every benchmark on the custom test suite described in Section 4 and reject any obfuscated individual whose output differs from the original on any test case. Only behavior-preserving individuals are therefore eligible to enter the Pareto front. We note that this validation is dynamic and thus as strong as the test suite; we discuss this limitation in Section 6.

3.1.2 Traditional Techniques

- **Control Flow Flattening:** The body of a function is restructured into a state machine controlled by a central dispatcher loop [33].
- **Function Inlining:** A function call is replaced with the actual code from the function's body [32].

- **Function Outlining:** A segment of code is extracted into a new function, and the original location is replaced with a call to this new function [32].
- **Code Reordering:** Independent statements whose execution order does not affect program semantics are swapped [32].
- **Dead Code Insertion:** Semantically null code is inserted into the program to increase its size and complexity [32].

The **Evaluation** phase assesses the fitness of each individual. We state the optimization problem in full here. Let P_0 denote the original program and P an obfuscated individual. We treat obfuscation as the multi-objective problem of jointly maximizing potency and minimizing cost,

$$\mathbf{F}(P) = (f_{cc}(P), f_{nl}(P), f_{sim}(P), f_{rt}(P)), \quad (1)$$

optimized over the space of programs reachable from P_0 by sequences of the semantics-preserving mutations defined above. The four components are the objectives the MOEAs trade off; program length is not optimized but is used to group results for fair comparison (Section 4). Each objective is defined as follows.

- **Program Length:** The total number of nodes in the AST of P . Used as a normalization/grouping variable rather than as an optimization objective.
- **Cyclomatic Complexity** (f_{cc} , *maximize*): A measure of program complexity based on its control-flow graph (CFG), calculated as $M = E - N + 2B$, where E is edges, N is nodes, and B is the number of connected components. It is a proxy for the difficulty of statically reasoning about all execution paths; higher values are intended to raise an analyst's effort.
- **Nesting Level Complexity** (f_{nl} , *maximize*): The maximum nesting depth of control structures in the program (e.g., nested loops and conditionals), a proxy for the cognitive load of tracking deeply nested control logic.
- **Similarity** (f_{sim} , *minimize*): A measure of syntactic resemblance to the original, calculated as

$$f_{sim}(P) = 1 - \frac{edit(P_0, P)}{tokens(P_0) + tokens(P)}, \quad (2)$$

where $edit(\cdot)$ is the token-level edit distance and $tokens(\cdot)$ is the number of tokens. A value of 1 denotes an identical token stream and 0 a maximally different one; minimizing it frustrates clone- and diff-based recovery of the original.

- **Running Time** (f_{rt} , *minimize*): The median wall-clock execution time over 10 runs on the program's test inputs. This is the principal *cost* objective: obfuscation that slows the program is undesirable in deployment.

Why the objectives conflict (and how they relate). The potency objectives (f_{cc} , f_{nl} , lower f_{sim}) and the cost objective (f_{rt}) are inherently opposed: the transformations that add control-flow edges, nesting, and indirection to raise complexity and lower similarity also add instructions to execute, so pushing potency tends to inflate running time—this is exactly the trade-off that motivates a Pareto, rather than a single-objective, formulation. The potency objectives are *not* mutually redundant either: similarity is a layout/token-level measure, whereas cyclomatic complexity and nesting level are control-flow measures, so a transformation such as identifier renaming can sharply reduce similarity without changing either complexity metric. Cyclomatic complexity and nesting level, however, are both derived from the CFG and can be partially correlated (a transformation that adds nested branches raises both); we therefore retain them as distinct objectives and analyze their empirical correlation and the resulting Pareto trade-offs in Sections 5.4 and 5.5.

Scalarization for decomposition-based MOEAs. NSGA-II operates directly on the objective vector of Eq. 1 via Pareto dominance. MOEA/D and MOEA/D/ALT instead decompose the problem into scalar subproblems using a set of uniformly distributed weight vectors $\{\lambda^1, \dots, \lambda^m\}$. After rewriting all objectives in minimization form (negating the two maximization objectives), each subproblem j minimizes the Tchebycheff scalarizing function

$$g^{te}(P | \lambda^j, \mathbf{z}^*) = \max_{1 \leq i \leq 4} \lambda_i^j |f_i(P) - z_i^*|, \quad (3)$$

where $\mathbf{z}^*$ is the (dynamically updated) reference point holding the best value found so far for each objective, and objective values are min–max normalized to $[0, 1]$ so that no single objective dominates the scalarization due to scale. A newly generated solution replaces the incumbent of subproblem j whenever it yields a smaller g^{te} , which is the update rule whose neighborhood scope distinguishes MOEA/D from MOEA/D/ALT (Section 3.2).

3.2 Multi-objective Evolutionary Algorithms Implemented

We implemented and compared three distinct MOEAs within our framework.

3.2.1 NSGA-II

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) [21] is a classic domination-based MOEA. In our implementation, the initial population consists of identical copies of the original AST. In each generation, every individual undergoes mutation. The combined parent and offspring population is sorted into non-dominated fronts. A

new population is formed by selecting individuals from the best fronts, using a crowding distance metric to maintain diversity.

3.2.2 MOEA/D

The Multi-Objective Evolutionary Algorithm based on Decomposition (MOEA/D) [25] converts the single multi-objective problem into m scalar single-objective subproblems, one per weight vector λ^j , each minimizing the Tchebycheff function of Eq. 3. Subproblems whose weight vectors are close are expected to have similar optimal solutions, so each subproblem is assigned a *neighborhood* of its T nearest weight vectors (here $T = 5$; Table 4). In each iteration, for a given subproblem the algorithm (i) selects parent(s) from that subproblem's neighborhood, (ii) generates a candidate by mutation, (iii) updates the reference point $\mathbf{z}^*$, and (iv) replaces, within the neighborhood, every incumbent for which the candidate achieves a smaller scalarized value. Maintaining the current best solution of each subproblem yields an even spread of solutions across the Pareto front, while restricting replacement to the neighborhood is what keeps the search both local and diverse.

3.2.3 MOEA/D/ALT

This algorithm is our adaptation of MOEA/D, and the two differ in exactly one respect—the *scope of the replacement step*—which we isolate so that any performance difference can be attributed to it. In standard MOEA/D, a newly generated candidate may replace incumbents *only within the neighborhood* of the subproblem that produced it (a window of T subproblems). In MOEA/D/ALT (“ALT” for *all*-subproblem update), the candidate is instead compared against, and may replace, the incumbent of *every* subproblem j for which it improves $g^{te}(\cdot \mid \lambda^j, \mathbf{z}^*)$. Concretely, the inner replacement loop ranges over all m subproblems rather than over the T neighbors. The intuition is that a single high-quality solution is often good for many weight vectors, not just nearby ones, so broadcasting it globally is *intended* to accelerate the dissemination of good genetic material; the trade-off is reduced population diversity and a higher per-iteration cost ($\mathcal{O}(m)$ instead of $\mathcal{O}(T)$ comparisons), since aggressively replacing many subproblems with the same solution can cause premature loss of spread. Whether this broader update actually helps is an empirical question we answer in Section 5. The two algorithms share identical mutation operators, weight vectors, reference-point handling, and computational budget, so the comparison in Section 5 cleanly measures the effect of broadening the cooperative update.

4 Experiment Evaluation

This section describes our experiment settings and evaluation information.

4.1 Research Questions

Our evaluation is designed to answer the following research questions:

- **RQ1:** What is the impact of including our proposed class-based obfuscation techniques on the quality of the generated code compared to using only traditional techniques?
- **RQ2:** Among the multi-objective evolutionary algorithms (NSGA-II, MOEA/D, and our global-update variant MOEA/D/ALT), all using class-based techniques, which is most effective for obfuscation, and does broadening MOEA/D's cooperative update (MOEA/D/ALT) improve on standard MOEA/D?
- **RQ3:** How does our overall approach compare to established, open-source obfuscation tools in terms of the trade-off between obfuscation potency and performance?

4.2 Experiment Setup

This subsection describes our experiment setup.

4.2.1 Benchmark Programs

We selected 15 Python programs from LeetCode [34] to serve as our benchmark suite. The problems were chosen from the “Top 100 Liked Questions” list, taking the five problems with the highest acceptance rates from each difficulty category (Easy, Medium, Hard). The specific problems are listed in

Table 2 LeetCode Problem Set used for Benchmarking

Difficulty	Problem Name
Easy	Invert Binary Tree; Reverse Linked List;
	Binary Tree Inorder Traversal;
	Maximum Depth of Binary Tree;
	Pascal's Triangle
Medium	Permutations; Subsets; Rotate Image;
	Generate Parentheses;
	Kth Smallest Element in a BST
Hard	N-Queens; Trapping Rain Water;
	Reverse Nodes in k-Group;
	Merge k Sorted Lists;
	Find Median from Data Stream

Table 3 Characterization of the 15 benchmark programs prior to obfuscation: physical lines of code (LOC), number of classes, methods (functions defined inside a class), and top-level functions, together with the baseline cyclomatic complexity (CC) and nesting level (NL)

ID	Problem	LOC	Cls.	Meth.	Fn.	CC	NL
<i>Easy</i>							
e1	Invert Binary Tree	60	2	2	3	27	4
e2	Reverse Linked List	42	2	2	3	20	3
e3	Binary Tree Inorder Traversal	51	2	3	2	25	4
e4	Maximum Depth of Binary Tree	42	2	2	2	19	4
e5	Pascal's Triangle	22	1	1	0	17	4
<i>Medium</i>							
m1	Permutations	40	1	2	0	20	3
m2	Subsets	21	1	1	0	14	4
m3	Rotate Image	35	1	2	0	13	5
m4	Generate Parentheses	35	1	2	0	14	3
m5	Kth Smallest Element in a BST	50	2	3	2	22	4
<i>Hard</i>							
h1	N-Queens	38	1	1	1	26	5
h2	Trapping Rain Water	22	1	1	0	16	4
h3	Reverse Nodes in k-Group	69	2	3	3	28	3
h4	Merge k Sorted Lists	65	2	2	3	28	5
h5	Find Median from Data Stream	57	1	3	0	19	5
Min / Mean / Max		21/43/69	1/1.5/2	1/2.0/3	—	13/21/28	3/4.0/5

Table 2. We created custom test cases for each problem to verify semantic correctness.

To characterize the dataset, each instance is a self-contained, single-file Python solution to one algorithmic problem. The programs are deliberately small and structurally lean—on the order of a few dozen source lines each—and most define a single primary class (or a small handful of top-level functions together with an auxiliary data-structure class such as a linked-list or tree node), with low baseline cyclomatic complexity and shallow nesting prior to obfuscation. This is a common profile for source-level obfuscation benchmarks, and the modest baseline complexity is what makes the *relative* inflation introduced by obfuscation (reported throughout as ratios to the original) a meaningful measure. The presence of at least one user-defined class in every instance is a prerequisite for exercising the class-based operators, which is why an object-oriented benchmark suite was chosen. Table 3 reports the exact per-instance figures—physical lines of code, the number of classes, methods (functions defined inside a class), and top-level functions, and the baseline cyclomatic complexity and nesting level. The instances span 21–69 lines of code, contain one or two user-defined classes (mean 1.5) and one to three methods, and have low baseline cyclomatic complexity (13–28) and shallow nesting (3–5) prior to obfuscation, quantitatively confirming the lean profile described above. The full benchmark suite and test cases are additionally released in the artifact referenced in the Data Availability statement.

4.2.2 Environment and Parameters

Experiments were conducted on a virtual machine running Ubuntu 24.04 LTS, with an Intel Core i5-7500 CPU and 16 GB of RAM. The Python version was 3.12.3. Each MOEA configuration was run 10 times on each benchmark program for 20 generations. The MOEA parameters are in Table 4.

4.2.3 Comparison Methodology

To ensure a fair comparison, we analyze results based on a consistent level of code bloat. After each run, we group the resulting programs from the Pareto front by their program length ratio (obfuscated length / original length). We compare the average metrics of programs with length ratios around $1.5\times$, $2.0\times$, $2.5\times$, and $3.0\times$.

4.2.4 Open-Source Obfuscators for Comparison

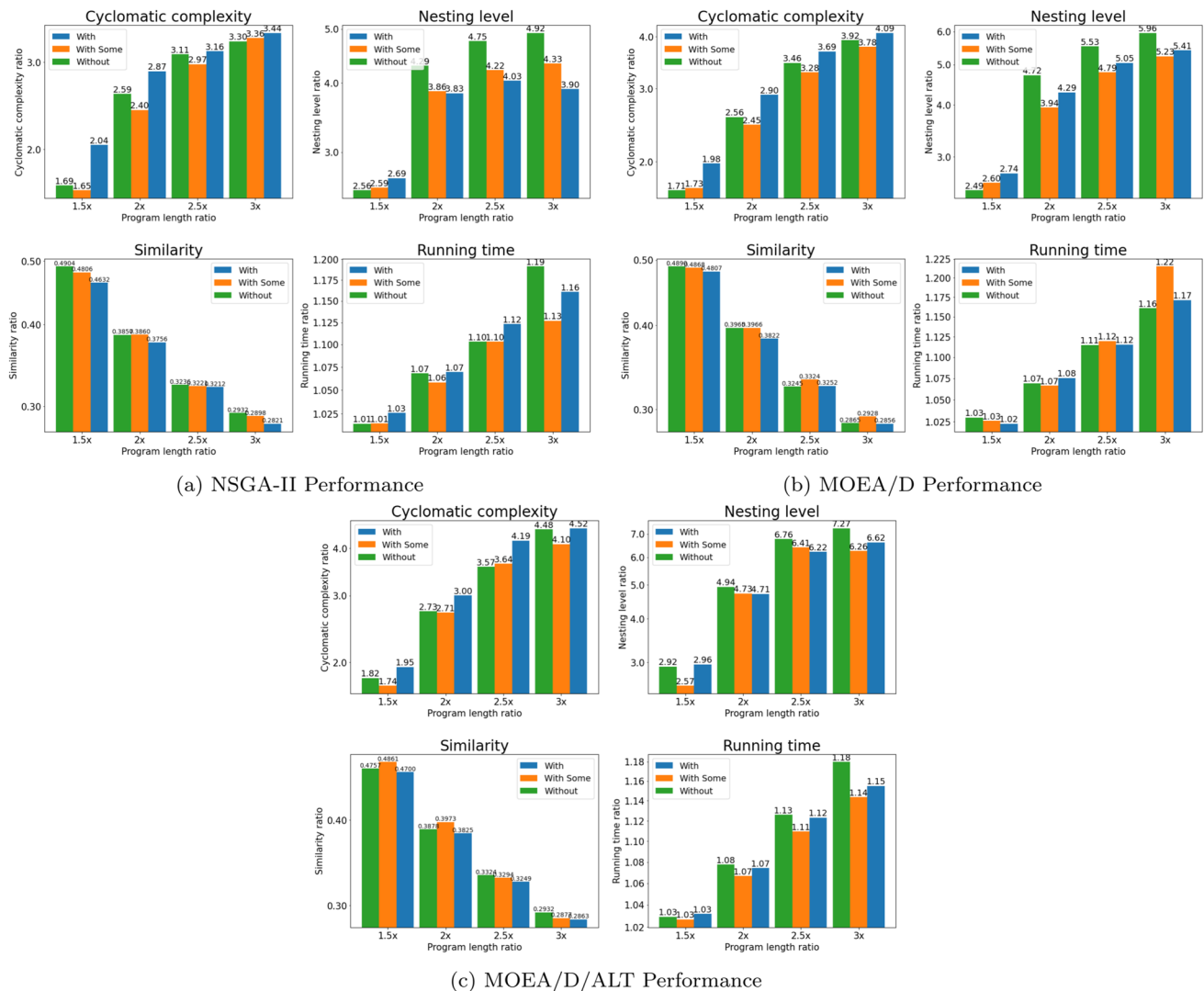
For RQ3, we compare our approach against three popular open-source Python obfuscators from GitHub that do not rely primarily on encryption: Intensio-Obfuscator [35], Opy [36], and pyminifier [37].

5 Experiment Results

This section presents our results. For statistical validation, we use the two-sided Wilcoxon rank-sum test with a significance

Table 4 Parameter Settings for the MOEAs

Parameter	NSGA-II	MOEA/D	MOEA/D/ALT
Number of runs	10	10	10
Generations per run	20	20	20
Initial population size	20	—	—
Maximum population size	100	—	—
Number of subproblems	—	70	70
Neighborhood size	—	5	5

**Fig. 2** Performance comparison for each MOEA with all class-based obfuscations ("With"), with a subset of class-based obfuscations ("WithSome"), and without any classbased obfuscations ("Without")

level of $\alpha = 0.05$. A p-value ≤ 0.05 indicates a statistically significant difference.

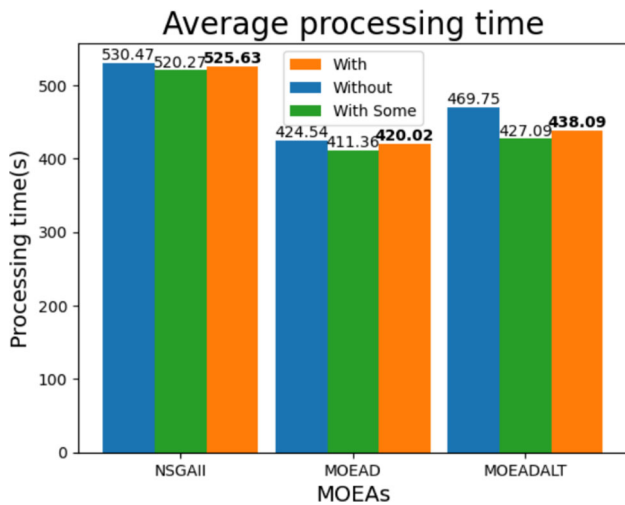
5.1 Effectiveness of Class-Based Obfuscation (RQ1)

To answer RQ1, we compared three configurations for each MOEA: one using all available obfuscation techniques

("With"), one using only a subset of the class-based operators together with the traditional techniques ("With Some"), and one using only traditional techniques ("Without"). Figure 2 visualizes the performance. A higher ratio is desirable for Cyclomatic Complexity and Nesting Level, while a lower ratio is better for Similarity and Running Time.

Table 5 P-values for comparing “With all” vs. “Without all” class-based techniques. Format: NSGA-II / MOEA/D / MOEA/D/ALT. Bold values are significant ($p \leq 0.05$)

Program Length Ratio	Cyclomatic Complexity	Nesting Level	Similarity	Running Time
1.5x	9.04E-06 / 3.84E-03 / 2.78E-02	0.23 / 3.83E-02 / 0.34	5.51E-06 / 2.09E-02 / 0.14	0.47 / 1.02E-02 / 0.52
2.0x	2.37E-04 / 3.21E-05 / 1.21E-03	1.00 / 0.99 / 0.80	5.19E-05 / 2.63E-06 / 2.59E-02	3.09E-02 / 0.92 / 0.76
2.5x	0.55 / 0.28 / 1.59E-09	1.00 / 1.00 / 0.99	2.63E-02 / 0.43 / 1.94E-03	1.00 / 0.90 / 0.68
3.0x	0.18 / 0.47 / 0.83	1.00 / 1.00 / 1.00	7.23E-09 / 0.44 / 2.63E-03	0.80 / 0.95 / 0.16

**Fig. 3** Average processing time (inseconds) for a single MOEA run under the three configurations “With”, “With Some”, “Without”

The “With” configuration consistently produces code with higher cyclomatic complexity and lower similarity than the “Without” configuration. This indicates class-based techniques are effective at increasing structural complexity and altering the code’s token sequence. Table 5 confirms that for most cases, class-based obfuscation provides a significant improvement in these metrics.

Interestingly, the impact on nesting level is often not statistically significant. This suggests that our class-based techniques achieve complexity by manipulating the object-oriented design (e.g., via inheritance or delegation) rather than by creating deeply nested control structures. This aligns with research showing that effective obfuscation should disrupt the “structural code patterns” that analysis tools rely on [13].

For running time, there is rarely a significant difference. This is expected, as transformations like class splitting or renaming are primarily structural and typically involve lightweight operations like adding pointers or redirection, which have minimal impact on the computational hot paths of a program’s execution. It demonstrates that the substantial gains in obfuscation potency from class-based techniques come at a negligible performance cost.

Finally, we analyzed the processing time of the MOEAs (Figure 3). The “Without” configuration took slightly longer, likely because class-based mutations are computationally less expensive than some traditional techniques like control-flow flattening.

5.2 Comparison of the MOEAs (RQ2)

To answer RQ2, we compare the three MOEAs with class-based techniques applied to *all* of them, so that the algorithm is the only variable. This isolates the effect of the algorithm, just as RQ1 isolated the effect of the techniques. As before, we group Pareto solutions by program length ratio and test differences with the two-sided Wilcoxon rank-sum test.

The results are in Figure 4 and Table 6. MOEA/D/ALT achieves higher potency than the dominance-based NSGA-II—significantly so on cyclomatic complexity and dissimilarity (1–similarity), and on nesting level at the largest ratio—with the advantage emerging at the higher program length ratios, at the cost of a modestly higher running time (the expected potency–cost trade-off). Because MOEA/D and MOEA/D/ALT are statistically indistinguishable (the only significant differences between them are a few isolated cells whose mean ratios are nearly identical), the same advantage over NSGA-II holds for MOEA/D; broadening the cooperative update to a global replacement (MOEA/D/ALT) therefore yields no systematic improvement over standard MOEA/D, consistent with the reduced-diversity trade-off anticipated in Section 3.2. We therefore attribute the advantage over NSGA-II to *decomposition-based* cooperation rather than to the breadth of the replacement step, and we retain MOEA/D/ALT as a competitive variant rather than a uniformly superior one.

5.3 Comparison with Open-Source Obfuscators (RQ3)

For RQ3, we compared our approach—a decomposition-based MOEA (MOEA/D/ALT) with class-based techniques, which performs comparably to MOEA/D (Section 5)—against three open-source obfuscators. For each tool, we obfuscated our benchmark suite and then found solutions

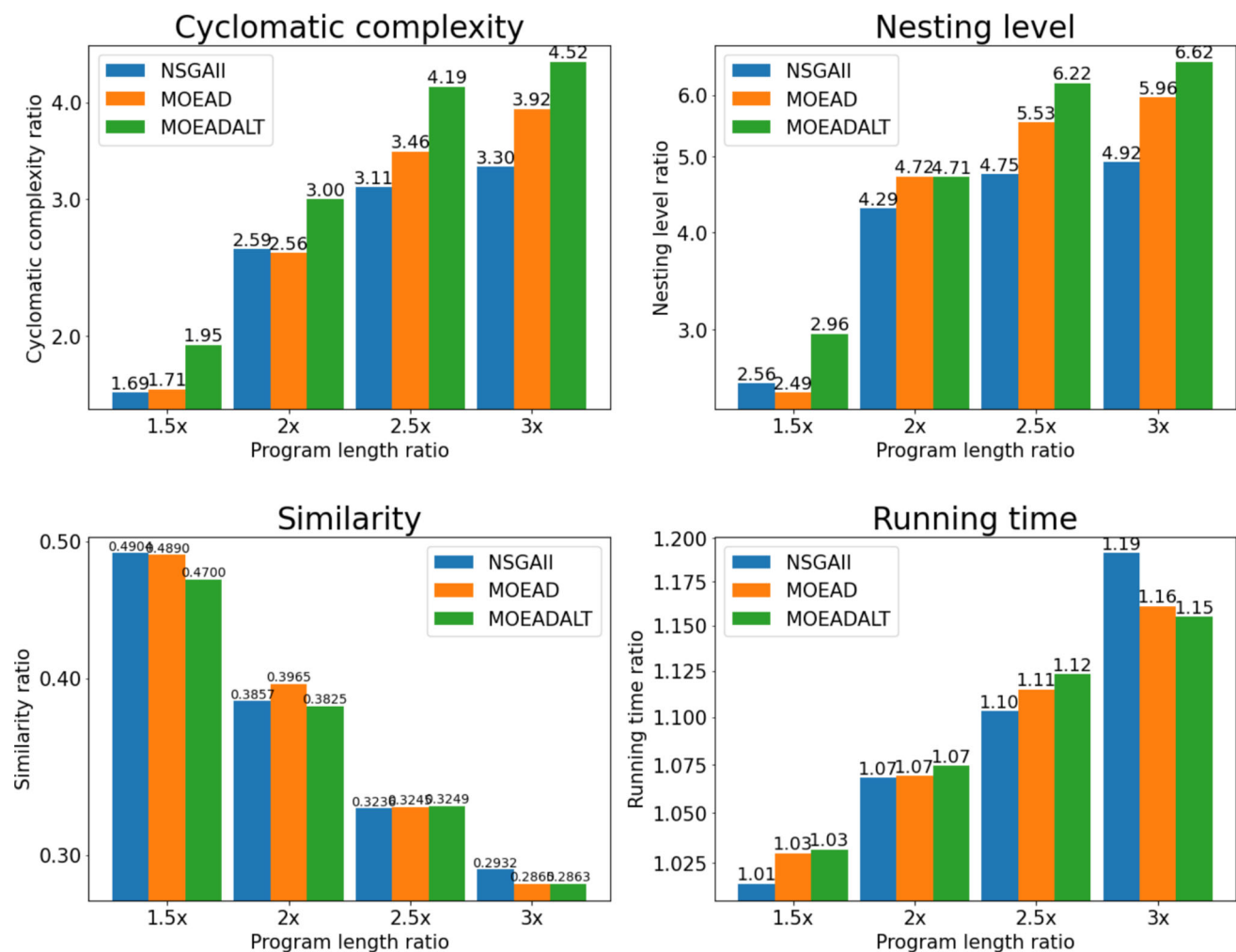


Fig. 4 Comparison of the three MOEAs, all using class-based techniques, grouped by program length ratio. Higher is more obfuscated for cyclomatic complexity and nesting level; lower is better for similarity and running time

Table 6 P-values (two-sided Wilcoxon rank-sum, as in RQ1) comparing MOEA/D/ALT against NSGA-II and MOEA/D, all with class-based techniques, so that the algorithm is the only difference. Format: $p(\text{vs NSGA-II}) / p(\text{vs MOEA/D})$. Bold values are significant ($p \leq 0.05$). The decomposition-based MOEA/D/ALT exceeds the dominance-based

NSGA-II on the potency objectives at the higher bloat ratios, whereas MOEA/D/ALT and MOEA/D are comparable—the few isolated significant cells between them correspond to negligible differences in the mean ratios

Ratio	Cyclomatic	Nesting Level	Similarity	Running Time
1.5x	0.28 / 5.89E-03	1.75E-02 / 0.93	0.45 / 0.37	0.12 / 0.12
2.0x	4.99E-02 / 0.57	0.73 / 0.21	2.93E-04 / 0.18	3.07E-07 / 7.69E-03
2.5x	0.43 / 0.20	0.45 / 0.20	1.91E-03 / 0.19	9.58E-05 / 0.70
3.0x	7.62E-03 / 0.82	1.23E-09 / 4.70E-02	1.24E-03 / 0.59	1.26E-09 / 0.77

from our algorithm's output with a comparable program length increase. Figure 5 presents the performance ratio of our method to the tool's performance. A value > 1.0 is better for complexity, while a value < 1.0 is better for similarity and running time.

As shown in Figure 5 and Table 7:

- **vs. Intensio-Obfuscator:** Our method achieves higher complexity with drastically lower running time (a total-runtime ratio of ~ 0.06 , i.e. about 94% lower) and lower similarity.
- **vs. Opy:** Our method overwhelmingly outperforms Opy on all potency metrics while also being more time-efficient.

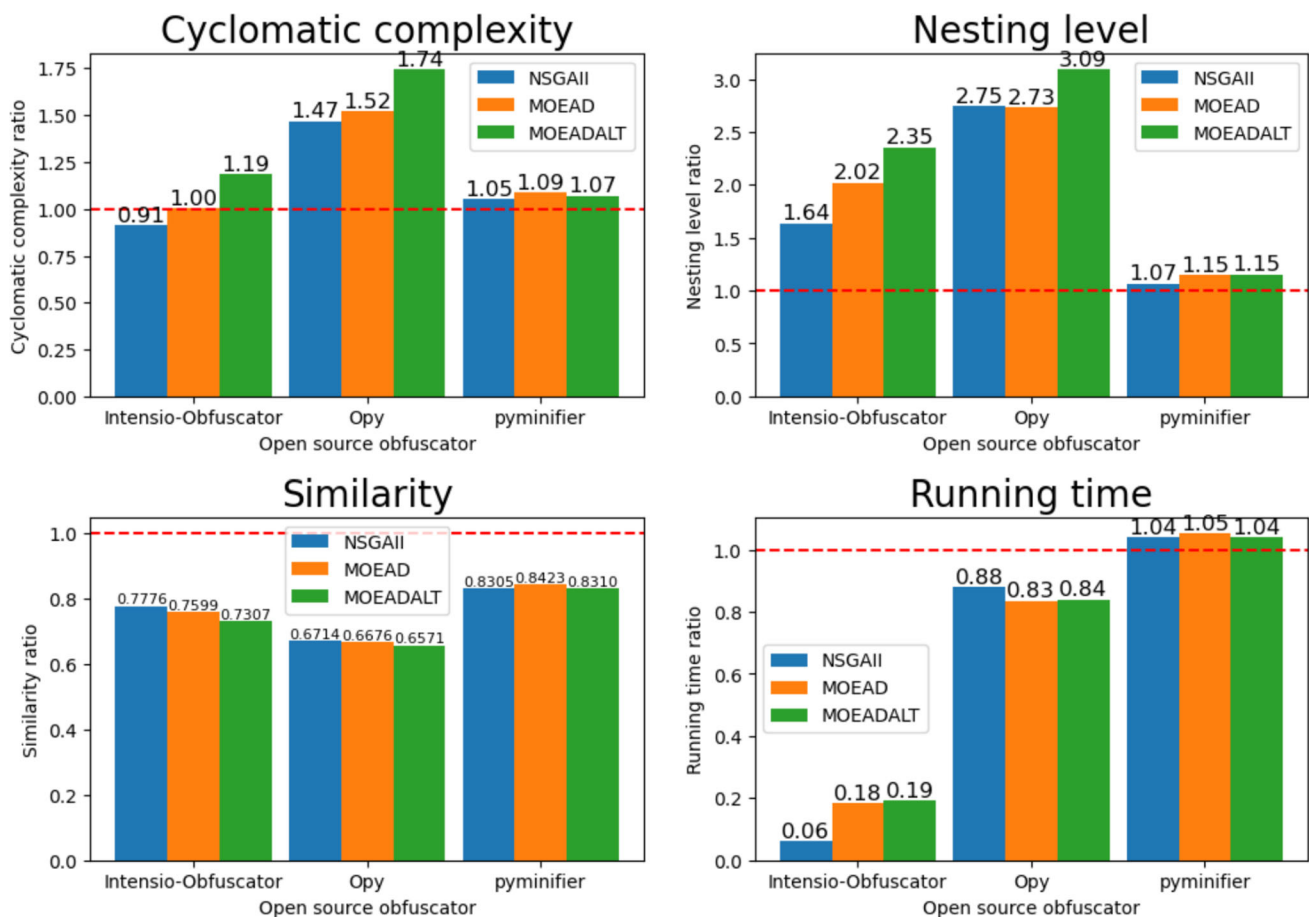


Fig. 5 Performance ratio of our MOEA approaches relative to three open-source obfuscators. The red line at 1.0 indicates parity. Values > 1 are better for complexity/nesting; values < 1 are better for similarity/time

Table 7 Performance metric changes of open-source obfuscators versus our results, at matched program-length inflation. Each value is the mean change over the benchmark suite; **bold** marks the better value

Obfuscator	Program Length Δ	Cyclomatic Δ	Nesting Level Δ	Similarity	Running Time Δ
Intensio-Obfuscator	+285.55%	+357.70%	+158.09%	0.3447	+2032.48%
Ours	+272.17%	+387.15%	+515.77%	0.2552	+26.28%
Opy	+66.49%	+20.76%	+0%	0.6841	+20.03%
Ours	+62.61%	+107.22%	+199.91%	0.4557	+4.21%
pyminifier	+13.90%	+0%	+0%	0.7303	+1.01%
Ours	+16.45%	+6.64%	+19.94%	0.5868	+4.65%

within each pairwise block (this comparison is descriptive—no significance test is applied in RQ3)

- **vs. pyminifier:** While pyminifier has the lowest overhead, our method achieves substantially better results on all potency metrics for a minimal increase in runtime.

These results show that our MOEA-based framework can generate obfuscated code with substantially higher structural potency at comparable or, in most cases, substantially lower performance overhead than that produced by standard, non-optimizing obfuscation tools.

5.4 Objective Correlation

We examine the internal structure of the objective space—whether the four objectives are redundant and how they trade off against one another—using the Pareto solutions obtained by the three MOEAs with class-based techniques (the runs underlying RQ1–RQ3). Table 8 reports the pairwise Spearman and Pearson correlations between the four objectives. To avoid spurious correlation induced by the

Table 8 Mean pairwise correlation between the four objectives, computed within each benchmark instance over all Pareto solutions of the three MOEAs (with class-based techniques) and averaged across the 15

instances. Each cell reports Spearman ρ / Pearson r . Objectives are oriented so that larger is more obfuscated (dissimilarity = 1 – similarity); running time is the cost

	Cyclomatic	Nesting	Dissimilarity	Running time
Cyclomatic	—			
Nesting	0.47/0.47	—		
Dissimilarity	0.82/0.76	0.39/0.44	—	
Running time	0.71/0.62	0.37/0.34	0.69/0.52	—

differing baselines of the 15 programs, each correlation is computed *within* an instance over all of that instance's Pareto solutions and then averaged across instances; Figure 6 visualizes the resulting Spearman matrix. Cyclomatic complexity and nesting level are the two objectives most likely to be redundant, but they are only moderately correlated ($\rho = 0.47$); they thus measure distinct aspects of structural potency, and keeping both as separate objectives is justified. Nesting level is the least entangled objective, yet it is only moderately—not negligibly—correlated with dissimilarity ($\rho = 0.39$) and running time ($\rho = 0.37$). The strongest association is between cyclomatic complexity and dissimilarity ($\rho = 0.82$): transformations that add control-flow structure also tend to move the token stream away from the original. Even this strongest pair, however, measures conceptually distinct properties—a control-flow-graph statistic versus a token-level layout statistic—and the two can move independently (identifier renaming lowers similarity without changing the control-flow graph, whereas control-flow flattening raises complexity with limited token change). The objectives are therefore positively correlated but not interchangeable, which is why we optimize them jointly on a Pareto front rather than collapsing them into a single aggregate score.

5.5 Pareto-Front Structure

Because the problem has four objectives, the front cannot be shown directly, so we give two complementary 2D views. Figure 7 shows the *true* Pareto front of a representative instance (Generate Parentheses): for each algorithm we take the non-dominated set in the full four-objective space and project those solutions onto three planes (cyclomatic complexity versus running time, dissimilarity versus running time, and the two potency objectives against each other). The complexity–time and dissimilarity–time panels show the expected potency–cost trade-off: both decomposition-based variants (MOEA/D and MOEA/D/ALT) reach higher complexity and dissimilarity than NSGA-II, at the cost of a somewhat higher running time. Some markers sit off the lower envelope of a panel: these solutions are non-dominated in four dimensions but look dominated once projected, an

unavoidable effect of viewing a high-dimensional front in 2D. Figure 8 summarizes the same behavior across the suite, overlaying the normalized trade-off frontiers pooled over all 15 instances (as ratios to the original program); the two decomposition-based variants form the outer (more potent) frontier while NSGA-II remains inner. The MOEA/D and MOEA/D/ALT frontiers overlap, consistent with RQ2, where the two are statistically indistinguishable on the potency objectives. Higher-dimensional visualizations and per-instance fronts for the remaining programs are included in the artifact.

6 Threats to Validity and Limitations

We discuss the principal limitations of our study and the threats they pose to the validity of our conclusions.

Proxy metrics versus true resilience. Our potency objectives—cyclomatic complexity, nesting level, and token similarity—are widely used *proxies* for the difficulty of reverse engineering, but they are not direct measurements of it. High cyclomatic complexity does not necessarily entail resilience against a skilled human analyst, and in particular modern, AI-assisted and pattern-based decompilers are increasingly effective at simplifying mechanically generated complexity such as control-flow flattening or dead code. Our results should therefore be read as evidence that the framework efficiently maximizes accepted structural-complexity proxies at low performance cost, not as a claim of resilience against any specific (especially learning-based) adversary. Establishing the latter requires an attacker model and human- or tool-in-the-loop deobfuscation experiments, which we identify as important future work; the framework's metric-agnostic design means a learned “deobfuscation-resistance” objective could be substituted for, or added to, the current proxies without altering the search engine.

Correlation among objectives. As noted in Section 3, cyclomatic complexity and nesting level are both CFG-derived and could in principle correlate and bias the search toward one structural dimension; the analysis in Section 5.4 (Table 8) shows this is mild in practice ($\rho = 0.47$, and no pair of objectives approaches collinearity). A sensitivity study

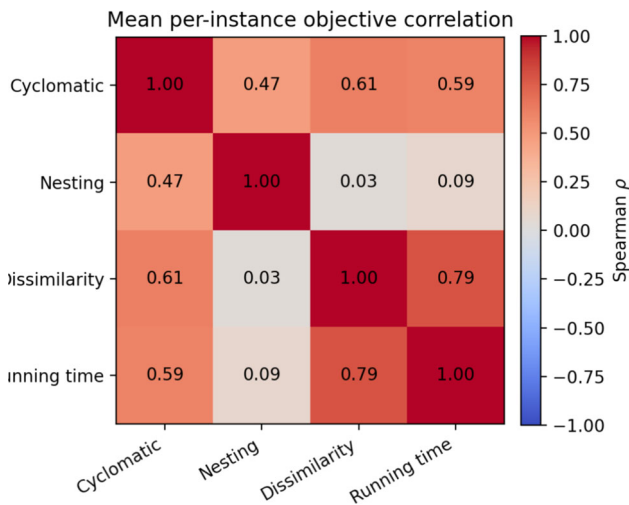


Fig. 6 Mean per-instance Spearman correlation between the four objectives across all obtained Pareto solutions. All pairs are positively but only moderately-to-strongly correlated (e.g., cyclomatic complexity vs. nesting level $\rho = 0.47$; the strongest, cyclomatic complexity vs. dissimilarity, $\rho = 0.82$), with no pair approaching collinearity

across a wider problem population remains valuable future work, and the per-solution objective values are included in the artifact.

Single-language evaluation. Although the framework’s optimization engine is language-agnostic in its architecture, it searches over an AST and delegates language-specific details to the operators and the parser/unparser; it is implemented and validated only for Python. Python is dynamically typed and interpreted, and several class-based operators rely on behaviors—such as permissive method resolution and the absence of compile-time type checking—that do not transfer unchanged to statically typed, ahead-of-time-compiled languages like Java or C++. In those settings, operators such as Polymorphic Class Renaming or Class Merging would require type-compatibility analysis and possibly signature

reconciliation to remain valid. Our cross-language “adaptability” is therefore a property of the architecture and a direction for future work, not an empirically validated result.

Behavior-preservation guarantees are test-bounded. While each transformation is a semantics-preserving refactoring by construction, our automated acceptance of a candidate ultimately depends on *dynamic* validation against the per-problem test suite (Section 3). Behavior preservation is therefore guaranteed only to the extent that the tests are adequate; a transformation that diverges only on unexercised inputs could in principle survive. We mitigate this with hand-written tests covering edge cases for each problem; broadening input coverage with automated test-input generation (e.g., fuzzing or property-based testing) would further reduce this risk, while formal verification of the operators would provide the strongest guarantee. Both are left to future work.

Benchmark scope. The benchmark suite consists of 15 small, self-contained algorithmic programs. This controlled setting isolates the effect of the transformations and supports statistical testing, but it does not establish how the framework scales to large, multi-module industrial codebases, where inter-module dependencies and build systems introduce additional constraints. Evaluation on larger real-world software is needed before drawing conclusions about industrial deployment.

Statistical analysis. Our significance tests are computed per length-ratio bucket and per objective, so each table involves many comparisons. We report uncorrected p -values and accordingly draw conclusions from *consistent patterns across length ratios* rather than from isolated significant cells; we do not apply a formal multiple-comparison correction (e.g., Holm) or report standardized effect sizes (e.g., Cliff’s δ), both of which would further sharpen the analysis. Relatedly, once solutions are grouped by length ratio the per-cell sample is small, so a non-significant result—such as

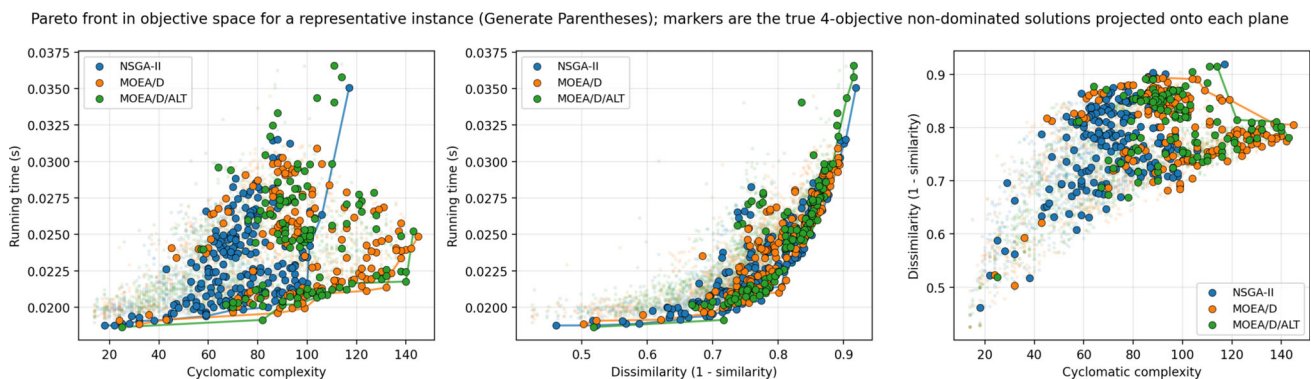


Fig. 7 Pareto front in objective space for a representative instance (Generate Parentheses). Markers are the *true* four-objective non-dominated solutions of each MOEA (with class-based techniques) projected onto

each plane; faint points are all obtained solutions; the line traces the 2D non-dominated envelope of the projected front. Up-and-to-the-right is more potent; lower is cheaper (running time)

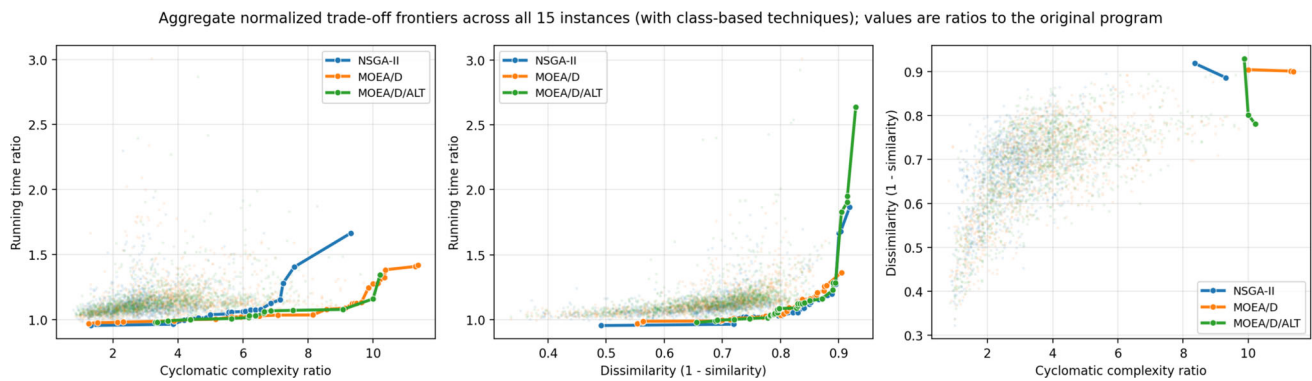


Fig. 8 Aggregate normalized trade-off frontiers across all 15 instances for the three MOEAs (with class-based techniques), expressed as ratios to the original program. Light points are the pooled solutions; bold

curves trace each algorithm's non-dominated frontier in the projection. Up-and-to-the-right is more potent; lower is cheaper (running time)

the limited effect of class-based operators on nesting level, or the parity between MOEA/D and MOEA/D/ALT—should be read as the absence of a difference detectable at our statistical power, not as proven equivalence. Larger benchmark populations, effect-size reporting, and multiplicity control are natural refinements for future work.

7 Conclusion and Future Work

In this paper, we introduced MOCCO, a novel framework for source code obfuscation that leverages Multi-Objective Evolutionary Algorithms to navigate the complex trade-offs between obfuscation potency and performance cost. Two aspects distinguish our approach from prior search-based obfuscation work, which has operated almost entirely at the statement, control-flow, or compiler-IR level. First, we introduce *class-based* obfuscation: a new family of operators that target a program's object-oriented *structure*—how its members are partitioned into classes and how methods are dispatched—a dimension largely neglected by modern tools. Second, our search engine operates directly on the source-level AST and is metric-agnostic, decoupling the optimization from any particular obfuscation operator or potency proxy. Our key contributions are these class-based techniques, the AST-level MOEA framework (including the novel MOEA/D/ALT variant), and a comprehensive, reproducible empirical evaluation.

Our findings address our research questions as follows. **(RQ1)** Including class-based techniques increases cyclomatic complexity and reduces code similarity—significantly so for the majority of cases, most clearly at the smaller length

ratios where the per-unit-length advantage is largest—with no consistent running-time penalty. **(RQ2)** With class-based techniques applied to all algorithms, the decomposition-based MOEAs (MOEA/D and MOEA/D/ALT) achieve higher structural potency than NSGA-II at a modestly higher running-time cost, whereas the two decomposition variants perform comparably—broadening the cooperative update (MOEA/D/ALT) does not significantly improve on standard MOEA/D. **(RQ3)** At comparable length inflation, MOCCO attains higher structural complexity than three popular open-source obfuscators while incurring comparable or, in most cases, substantially smaller performance overhead. We measure potency with established structural-complexity metrics; quantifying resilience against concrete (including AI-assisted) adversaries and extending MOCCO beyond Python are the natural next steps (Section 6).

For future work, several promising avenues exist. The introduction of a robust, AST-based crossover operator could further enhance the exploratory power of the evolutionary algorithms. Second, the framework's AST-based design makes it inherently adaptable to other programming languages. Future work could involve extending the implementation to support statically-typed, object-oriented languages like Java or C++, which would require developing language-specific parsers and transformation rules while preserving the core optimization engine. Finally, the integration of Large Language Models (LLMs) could dramatically improve the scalability and adaptability of the framework. An LLM could be trained to suggest or even generate new, complex obfuscation transformations, or to assist in porting the framework to new target languages, further advancing the state of the art in automated software protection.

Appendix A Examples of the Obfuscated Code Generated by Our Work

With 1.5x program length

Below is the obfuscated code of Generate Parentheses. The original problem description is available on <https://leetcode.com/problems/generate-parentheses/>.

```

1 from binascii import a2b_hex
2 import sys
3
4 class MhffvG:
5
6     def CcJrDf(GbhTYm, wXBbuP):
7         dtOiRJ = 97909152
8         while dtOiRJ != 26874185:
9             if dtOiRJ == 72572245:
10                 dtOiRJ = 22762281
11                 vTMxPJ = 0
12             elif dtOiRJ == 22762281:
13                 dtOiRJ = 47441096
14                 FwCvLK = 0
15             elif dtOiRJ == 97909152:
16                 dtOiRJ = 81005819
17                 fvAyoq = []
18             elif dtOiRJ == 81005819:
19                 dtOiRJ = 72572245
20                 owFseV = a2b_hex(b'').decode()
21             elif dtOiRJ == 41466100:
22                 dtOiRJ = 26874185
23                 return fvAyoq
24             elif dtOiRJ == 47441096:
25                 GbhTYm.HqqZUR(fvAyoq, owFseV,
26                 ↪ vTMxPJ, FwCvLK, wXBbuP)
27                 dtOiRJ = 41466100
28
29 class rLeifD(MhffvG):
30
31     def HqqZUR(GbhTYm, fvAyoq, owFseV, vTMxPJ,
32     ↪ FwCvLK, wXBbuP):
33         if FwCvLK == wXBbuP and vTMxPJ == 0:
34             fvAyoq.append(owFseV)
35             return
36         if FwCvLK < wXBbuP:
37             owFseV += a2b_hex(b'28').decode()
38             vTMxPJ += 1
39             FwCvLK += 1
40             GbhTYm.HqqZUR(fvAyoq, owFseV,
41             ↪ vTMxPJ, FwCvLK, wXBbuP)
42             owFseV = owFseV[:-1]
43             vTMxPJ -= 1
44             FwCvLK -= 1
45         if vTMxPJ:
46             owFseV += a2b_hex(b'29').decode()
47             vTMxPJ -= 1
48             GbhTYm.HqqZUR(fvAyoq, owFseV,
49             ↪ vTMxPJ, FwCvLK, wXBbuP)
50             owFseV = owFseV[:-1]
51             vTMxPJ += 1
52
53 if __name__ ==
54     ↪ a2b_hex(b'5f5f6d61696e5f5f').decode():
55     AueROT = 18520974
56     while AueROT != 74843595:
57         if AueROT == 85354720:
58             AueROT = 92865025
59             jaqksu = [int(VuiqZA) for VuiqZA
60             ↪ in jaqksu.split()]
61             elif AueROT == 18520974:
62                 AueROT = 85354720
63                 with open(sys.argv[1],
64                 ↪ a2b_hex(b'72').decode()) as gEOBFV:
65                     jaqksu = gEOBFV.read()
66             elif AueROT == 92865025:
67                 AueROT = 74843595
68                 for VuiqZA in jaqksu:
69                     dhaBmK = rLeifD()
70                     owFseV = dhaBmK.CcJrDf(VuiqZA)

```

```
print(owFseV)
```

Listing 1 example of obfuscated code with 1.5x program length (Problem: Generate Parentheses)

With 2x program length

Below is the obfuscated code of NQueens. The original problem description is available on <https://leetcode.com/problems/n-queens/>.

```

1 from binascii import a2b_hex
2 import sys
3
4 class iOufXB:
5
6     def __init__(fUovOn, xdVfZd):
7         fUovOn.xdVfZd = xdVfZd
8
9     def ioLLvn(fUovOn, JRerTT, IkhqtF, lHprGT):
10         if len(JRerTT) == IkhqtF:
11             qyBwGW = 37605027
12             while qyBwGW != 12864568:
13                 if qyBwGW == 70393767:
14                     qyBwGW = 27465456
15                     for ZixRHe in JRerTT:
16                         OwZmpR = a2b_hex(b'').decode()
17                         for FwSkSC in range(IkhqtF):
18                             if FwSkSC == ZixRHe:
19                                 OwZmpR +=
20                                 ↪ a2b_hex(b'51').decode()
21                             else:
22                                 OKvmpe = 422139400502
23                                 OwZmpR +=
24                                 ↪ a2b_hex(b'2e').decode()
25                                 QTDno.append(OwZmpR)
26                             elif qyBwGW == 27465456:
27                                 qyBwGW = 87601776
28                                 if (958666347 + 228458311) % 37508
29                                 ↪ == 33123:
30                                     zmbSrt = 219015294448159917
31                                     else:
32                                         zmbSrt = 866847965005351688
33                                     elif qyBwGW == 87601776:
34                                         kEIfHw = 68349020
35                                         while kEIfHw != 40589351:
36                                             if kEIfHw == 68349020:
37                                                 kEIfHw = 36081386
38                                                 qyBwGW = 54859168
39                                             elif kEIfHw == 36081386:
40                                                 kEIfHw = 40589351
41                                                 lHprGT.append(QTDno)
42                                             elif qyBwGW == 37605027:
43                                                 qyBwGW = 70393767
44                                                 QTDno = []
45                                             elif qyBwGW == 54859168:
46                                                 qyBwGW = 12864568
47                                                 return
48                     else:
49                         kMpQpk = a2b_hex(b'65664169535a4243574855664
50                         ↪ d507651796166535073526c62704f4b544f476e72495
51                         ↪ 05155704a456f6c7348627a58776554436d674763784
52                         ↪ f574d4c56776570576a64535256446257684e4657724
53                         ↪ 475797a797678725a755466786741437349526971567
54                         ↪ 171427576415044756c766f57476d7a457a487949657
55                         ↪ 56a6f475a51644b434c4a74464d5755797a6b7061554
56                         ↪ 356766e496d4d77686b664d51564a454a5448546d4c5
57                         ↪ 96c4c454661624467464d595270754f5442424447684
58                         ↪ e51466658574c6f7079626b62556759614f744b6e794
59                         ↪ 14d7469514c73666d79616f7266477961627a70434e6
60                         ↪ 26b716771546177457a766c764c59455658697843644
61                         ↪ d53').decode()
62                     yLJwen = len(JRerTT)
63                     if (552460798 + 598346823) % 65639 == 22253:
64                         BGHinc = 330563163315344754
65                     else:
66                         BGHinc = -52805987267396525
67                     for fOmQcf in range(IkhqtF):

```

```

53         fygArs = True
54         for FwSkSC in range(yLJwen):
55             if JRerTT[FwSkSC] == fOmQCf or yLJwen
56             ↪ - FwSkSC == abs(fOmQCf - JRerTT[FwSkSC]):
57                 fygArs = False
58                 break
59             if fygArs:
60                 JRerTT.append(fOmQCf)
61                 if (898662518 + 61229891) % 98337 ==
62                 ↪ 66494:
63                     yOoqVO = 55025008022925538
64                 else:
65                     yOoqVO = 803845221706228443
66                     fUovOn.xdVfZd.GrbLsJ(JRerTT, IkhqtF,
67                     ↪ lHpRGt)
68                     JRerTT.pop()
69
70 class kEqmOP:
71
72     def __init__(fUovOn):
73         fUovOn.VwiWuz = iOuFXB(fUovOn)
74
75     def GrbLsJ(fUovOn, JRerTT, IkhqtF, lHpRGt):
76         return fUovOn.VwiWuz.ioLLvn(JRerTT, IkhqtF,
77         ↪ lHpRGt)
78
79 def QYxWoE(IkhqtF):
80     lHpRGt = []
81     JRerTT = []
82     YqQgKj = kEqmOP()
83     gHkdWk = YqQgKj
84     gHkdWk.GrbLsJ(JRerTT, IkhqtF, lHpRGt)
85     return lHpRGt
86
87 if __name__ == a2b_hex(b'5f5f6d61696e5f5f').decode():
88     with open(sys.argv[1], a2b_hex(b'72').decode()) as
89     ↪ xONoTH:
90         SMcSVJ = xONoTH.read()
91         SMcSVJ = [int(ZixRHe) for ZixRHe in SMcSVJ.split()]
92         for ZixRHe in SMcSVJ:
93             print(QYxWoE(ZixRHe))

```

Listing 2 example of obfuscated code with 2x program length (Problem: NQueens)

With 2.5x program length

Below is the obfuscated code of Reverse Nodes in k-Group. The original problem description is available on <https://leetcode.com/problems/reverse-nodes-in-k-group/>.

```

1 from binascii import a2b_hex
2
3 class CEXPUn:
4
5     def __init__(DGqSJY, MdCkbw, zyCqlj=a2b_hex(b'69
6     ↪ 54484473484852756a577a4f42657a776d7743474b5
7     ↪ 6775a42587173637a766f67516a4f6e626d714d6b75
8     ↪ 7a6c4d4169616976646b6f745171416a66474d51426
9     ↪ 26c4d58416d706c636149787a467470696f5a415958
10    ↪ 6559564e77745746764f506d79').decode(),
11    ↪ OfjAyc=a2b_hex(b'726573').decode()):
12        if MdCkbw:
13            DGqSJY.NTRCUX = zyCqlj
14            if (46318495 + 333062941) % 11811 == 10792:
15                juLxIk = 15426974167393795
16            else:
17                juLxIk = -108785519688504456
18                DGqSJY.ELZjTM = None
19                DGqSJY.WVduEG = LZVPGC(DGqSJY)
20                DGqSJY.gTrPDA = xLGraw(DGqSJY)
21            else:
22                DGqSJY.ZRkApw = OfjAyc
23
24    def hgbNuS(DGqSJY, RnebDX):
25        return DGqSJY.ZRkApw.EKxKBM.dpHgMp(RnebDX)
26
27    def zyKPPc(DGqSJY, NlkwOC, pxDrFF):

```

```

22         return DGqSJY.ZRkApw.EKxKBM.MCLyZD(NlkwOC,
23         ↪ pxDrFF)
24
25 def KgJfTF(NlkwOC):
26     KyZbdf = []
27     while NlkwOC:
28         KyZbdf.append(NlkwOC.NTRCUX)
29         NlkwOC = NlkwOC.ELZjTM
30     return KyZbdf
31
32 def qdJRwb(eORMQA):
33     bMCPeQ = CEXPUn(1, -1, 0)
34     OLgaim = bMCPeQ
35     CADZxC = a2b_hex(b'6b6c4c6d507854584554534763767
36     ↪ a4f5a6949576c79415150737165534b63596e504169
37     ↪ 4d767072496663534e446763756a5779777a5565485
38     ↪ 95467785a4350504b615657785471536f5276455164
39     ↪ 4d63555a6352766375696b567a497a7768575167696
40     ↪ e6b4e704f6b4845474f734c486758675545576e7a59
41     ↪ 5456726471526b6f5864444a6e5843694b614f55617
42     ↪ 846444e66755579774d6c6e764f495a79766a4c5a70
43     ↪ 696c71744c67485041554d47424f416c494a6c4a4e6
44     ↪ 745617066684d5a4b776c506b4e4162796465655259
45     ↪ 73484b6566655a667263515362624e6848755543615
46     ↪ 26970536e445349756862504f537a41574b4f4d7a50
47     ↪ 6464754272').decode() + a2b_hex(b'4f4e78414
48     ↪ b74756d5a6651684366767044714868486243765a6e
49     ↪ 51434774795a7457504c516b4249476f50444974464
50     ↪ 1476847477741504557657654744d41694349464441
51     ↪ 70736d4c696550417952434a6e71537479476763737
52     ↪ 042494c49416856597471724564664b724e716e5050
53     ↪ 544a6d496d704b6d767779757a49514d476956534b5
54     ↪ 66d776d467a537853714463454a536b7763747484e72
55     ↪ 4b46485a666750695952524a504c4346416b566c727
56     ↪ a4b427377667a6b684767534694d7279484f636450
57     ↪ 704e774c42674c464361527267796f704a484852785
58     ↪ 3556d54676a4e4a434551596b737168666863587656
59     ↪ 50485078517656426e7a45505a6549').decode()
60
61 RnebDX = OLgaim
62 yXDUGd = RnebDX
63
64 for CPbwMg in eORMQA:
65     kQmClB = CEXPUn(1, CPbwMg, 1000000000)
66     ZduWDg = kQmClB
67     yXDUGd.ELZjTM = ZduWDg
68     yXDUGd = yXDUGd.ELZjTM
69
70 return RnebDX.ELZjTM
71
72 def tfHpdL(eORMQA):
73     pxDrFF = eORMQA[0]
74     eORMQA = eORMQA[1:]
75     if (108976046 + 421224323) % 92589 == 51314:
76         rmcNtU = 45903361199566858
77     else:
78         rmcNtU = -165554151685014213
79     fVYIaS = eORMQA
80     FWbSjg = CEXPUn(1, -1, 1000000000)
81     SUOISP = FWbSjg
82     HgelaR = SUOISP
83     fohUkk = HgelaR
84     for HSSHcU in fVYIaS:
85         vxJTfU = CEXPUn(1, HSSHcU, a2b_hex(b'6954484
86         ↪ 473484852756a577a4f42657a776d7743474b56775a4
87         ↪ 2587173637a766f67516a4f6e626d714d6b757a6c4d4
88         ↪ 169616976646b6f745171416a66474d5142626c4d584
89         ↪ 16d706c636149787a467470696f5a4159586559564e7
90         ↪ 7745746764f506d79').decode())
91         JzcgGK = vxJTfU
92         fohUkk.ELZjTM = JzcgGK
93         fohUkk = fohUkk.ELZjTM
94     PfzrkW = HgelaR.ELZjTM
95     kispXE = PfzrkW
96     RnebDX = kispXE
97     FhRROi = mFVGjD()
98     VNpuJH = FhRROi
99     vllRBQ = VNpuJH
100    RnebDX = vllRBQ.CxKnyK(RnebDX, pxDrFF)
101    cUYFoI = RnebDX
102    TJauCz = []
103    while cUYFoI:
104        TJauCz.append(cUYFoI.NTRCUX)
105        cUYFoI = cUYFoI.ELZjTM
106    WTotZc = TJauCz

```

```

74     xiyNfO = WTotZc
75     return xiyNfO
76 import sys
77
78 class LZVPGC:
79
80     def __init__(DGqSJY, HRZWKG):
81         DGqSJY.HRZWKG = HRZWKG
82
83 class xlGraw:
84
85     def __init__(DGqSJY, olGUTV):
86         DGqSJY.olGUTV = olGUTV
87
88 class ALZGZg:
89
90     def __init__(DGqSJY, gsHEBY):
91         DGqSJY.gsHEBY = gsHEBY
92
93     def dpHgMp(DGqSJY, RnebDX):
94         yXDUGD = RnebDX
95         EiLOyP = None
96         while yXDUGD.ELZjTM != None:
97             LjJjbd = yXDUGD.ELZjTM
98             yXDUGD.ELZjTM = EiLOyP
99             EiLOyP = yXDUGD
100             yXDUGD = LjJjbd
101         yXDUGD.ELZjTM = EiLOyP
102         return (yXDUGD, RnebDX)
103
104     def MCLYzD(DGqSJY, NlkwoC, pxDrFF):
105         if pxDrFF == 1:
106             return NlkwoC
107         yXDUGD = GbYwOT = NlkwoC
108         EiLOyP = QVghSC = None
109         while yXDUGD:
110             GbYwOT = yXDUGD
111             TGxpdj = 1
112             while GbYwOT.ELZjTM and TGxpdj < pxDrFF:
113                 TGxpdj += 1
114                 GbYwOT = GbYwOT.ELZjTM
115             if TGxpdj < pxDrFF:
116                 EiLOyP.ELZjTM = yXDUGD
117                 break
118             elif (475454910 + 917791370) % 90767 ==
119                 3483:
120                 EkDjfo = 436368413222126700
121             else:
122                 EkDjfo = -616283627403368800
123                 LjJjbd = GbYwOT.ELZjTM
124                 GbYwOT.ELZjTM = None
125                 dWOeha, pCDDol =
126                 DGqSJY.gsHEBY.NnfBbe(yXDUGD)
127                 if EiLOyP == None:
128                     QVghSC = dWOeha
129                     if (964245871 + 141791022) % 92266 ==
130                         90445:
131                             OpffIE = 136721407508370162
132                         else:
133                             fobdnF = 469607393014
134                             OpffIE = 909665405820744157
135                             EiLOyP = pCDDol
136                             ePHsUl =
137
138         a2b_hex(b'6572455976497a6b6a75694c
139         67454458556d6b506e557473534c7477417569544572
140         657178494856526a6779436a6e79756e496e444d4371
141         784767515a6278576c51477551626a514a4741765243
142         6c6b46766d43527a504b676a6a4d6846516168444d6b
143         6e52414979474e795971657a5851706a7444524c6c65
144         416b4b527443356f6b6b56747478456b644f6566706f
145         416a654a694c4e6e7456456d4a477147434670445449
146         7a747a52665a634b7062497552726b554b7163656f72
147         52564c43657367596c50495668726b547677496f7454
148         6353566274746a556378546b5448737367414f756c54
149         4f6171496d7577785652785469686b6a575076724b6a
150         586d').decode() + a2b_hex(b'4b57465949734572
151         53514d477a567077654e5a4f6e5161514f6943514c63
152         787461456c517373736c586271506c71795867716a6f
153         6453494153767657796d706659445841674c4f456554
154         4a48724457736371466850475371627765456b56596f
155         4e4457665370714c4a68764b5a575058595541637976
156         7156556366744671677641616a4c5775634b4d59585a

```

```

134         yXDUGD = LjJjbd
135     else:
136         EiLOyP.ELZjTM = dWOeha
137         EiLOyP = pCDDol
138         yXDUGD = LjJjbd
139     return QVghSC
140
141 class mFVGjD:
142
143     def __init__(DGqSJY):
144         DGqSJY.EKxKBM = ALZGZg(DGqSJY)
145         DGqSJY.dbTUfQ = CEXPUn(0,
146         a2b_hex(b'695448447348
147         4852756a577a4f42657a776d7743474b56775a4258717
148         3637a766f67516a4f6e626d714d6b757a6c4d41696169
149         76646b6f745171416a66474d5142626c4d58416d706c6
150         36149787a467470696f5a4159586559564e7774574676
151         4f506d79').decode(), DGqSJY)
152
153     def NnfBbe(DGqSJY, RnebDX):
154         return DGqSJY.dbTUfQ.hgbNuS(RnebDX)
155
156     def CxKnyK(DGqSJY, NlkwoC, pxDrFF):
157         return DGqSJY.dbTUfQ.zykPPc(NlkwoC, pxDrFF)
158 if __name__ == a2b_hex(b'5f5f6d61696e5f5f').decode():
159     oSnZxv = open(sys.argv[1], a2b_hex(b'72').decode())
160     gdtWKW = oSnZxv.readlines()
161     ZdubUu = [[int(CPbwMg) for CPbwMg in list(POhqsM.
162         strip().split(a2b_hex(b'2c').decode()))] for
163         POhqsM in gdtWKW]
164     for eORMqA in ZdubUu:
165         IgxfXr = eORMqA
166         sDjGrJ = IgxfXr[0]
167         IgxfXr = IgxfXr[1:]
168         if (108976046 + 421224323) % 92589 == 51314:
169             ULwLCP = 626702906635
170             PQBkdp = 45903361199566858
171         else:
172             PQBkdp = -165554151685014213
173         fVYIaS = IgxfXr
174         FTIBtd = CEXPUn(1, -1, 1000000000)
175         myHsMc = FTIBtd
176         HgelaR = myHsMc
177         fohUkk = HgelaR
178         for HSSHcU in fVYIaS:
179             vabfWc = CEXPUn(1, HSSHcU, None)
180             SCJuyX = vabfWc
181             fohUkk.ELZjTM = SCJuyX
182             fohUkk = fohUkk.ELZjTM
183         PfzrkW = HgelaR.ELZjTM
184         eCpUgh = PfzrkW
185         jClJG = eCpUgh
186         LZMAQc = mFVGjD()
187         PmfWTy = LZMAQc
188         BLftHz = PmfWTy
189         jClJG = BLftHz.CxKnyK(jClJG, sDjGrJ)
190         cUYFoI = jClJG
191         TJauCz = []
192         while cUYFoI:
193             TJauCz.append(cUYFoI.NTRCUx)
194             cUYFoI = cUYFoI.ELZjTM
195         WTotZc = TJauCz
196         kfBcep = WTotZc
197         hcIdOl = kfBcep
198         aVLRHx = hcIdOl
199         zfgPAd = aVLRHx
200         print(zfgPAd)

```

Listing 3 example of obfuscated code with 2.5x program length (Problem: Reverse Nodes in k-Group)

With 3x program length

Below is the obfuscated code of Invert Binary Tree. The original problem description is available on <https://leetcode.com/problems/invert-binary-tree/>.

```

1 from binascii import a2b_hex
2
3 class QQktPH:
4
5     def __init__(JRgCLZ, udmnYN, TQdFWH=1e-05,
6         ↪ vyKzME=1000000000):
7         if udmnYN:
8             JRgCLZ.OXJnhJ = TQdFWH
9         else:
10             JRgCLZ.NpAjtF = vyKzME
11             JRgCLZ.zzzYeR = None
12             JRgCLZ.EWqtmK = None
13
14     def lXRoZr(JRgCLZ, FNJSVf):
15         return JRgCLZ.OXJnhJ.qthFIy.HtpqqP(FNJSVf)
16
17 def IXAWxA(omihGa):
18     FNJSVf = None
19     pXtlGk = []
20     for FkjmW in omihGa:
21         IBjppv = QQktPH(0, 0, FkjmW)
22         if len(pXtlGk) == 0:
23             FNJSVf = IBjppv
24         elif pXtlGk[0].zzzYeR == None:
25             pXtlGk[0].zzzYeR = IBjppv
26         else:
27             pXtlGk[0].EWqtmK = IBjppv
28             pXtlGk.pop(0)
29             pXtlGk.append(IBjppv)
30     return FNJSVf
31
32 def hXgShE(omihGa):
33     arNKNz = omihGa
34     uxOTJq = None
35     dDEVHm = []
36     for MyWrWg in arNKNz:
37         PgVVQk = QQktPH(0, 1, MyWrWg)
38         if len(dDEVHm) == 0:
39             uxOTJq = PgVVQk
40         elif dDEVHm[0].zzzYeR == None:
41             dDEVHm[0].zzzYeR = PgVVQk
42         else:
43             dDEVHm[0].EWqtmK = PgVVQk
44             dDEVHm.pop(0)
45             dDEVHm.append(PgVVQk)
46     fAHdOD = uxOTJq
47     oVpVYg = fAHdOD
48     FNJSVf = oVpVYg
49     XoNgjY = EgPvXK()
50     eGjggK = XoNgjY
51     kiIogp = eGjggK
52     vLJFnf = kiIogp
53     FNJSVf = vLJFnf.BoxAnI(FNJSVf)
54     YvZlkY = FNJSVf
55     VoDuFT = []
56     PusylU = [YvZlkY]
57     while len(PusylU):
58         uDbdaY = PusylU[0]
59         PusylU.pop(0)
60         if uDbdaY == None:
61
62             ↪ VoDuFT.append(a2b_hex(b'4e6f6e65').decode())
63             continue
64             VoDuFT.append(uDbdaY.NpAjtF)
65             if uDbdaY.zzzYeR or uDbdaY.EWqtmK:
66                 PusylU.append(uDbdaY.zzzYeR)
67                 PusylU.append(uDbdaY.EWqtmK)
68         else:
69             if (467528573 + 722241577) % 44326 == 33427:
70                 xvwOpr = 337668573856079621
71             else:
72                 xvwOpr = -303049928976030600
73     yPXbpm = VoDuFT
74     JPOGZr = yPXbpm

```

```

73 TdsdWH = JPOGZr
74 return TdsdWH
75
76 def TWGZlS(FNJSVf):
77     omihGa = []
78     pXtlGk = [FNJSVf]
79     while len(pXtlGk):
80         IBjppv = pXtlGk[0]
81         pXtlGk.pop(0)
82         if IBjppv == None:
83
84             ↪ omihGa.append(a2b_hex(b'4e6f6e65').decode())
85             continue
86             omihGa.append(IBjppv.NpAjtF)
87             if IBjppv.zzzYeR or IBjppv.EWqtmK:
88                 pXtlGk.append(IBjppv.zzzYeR)
89                 pXtlGk.append(IBjppv.EWqtmK)
90     return omihGa
91
92 import sys
93
94 class apAdQa:
95
96     def __init__(JRgCLZ, JpdgZh):
97         if (891059137 + 624735259) % 71927 == 2639:
98             OYXeCz = 556676060738011483
99         else:
100             OYXeCz = 403692241793387688
101             JRgCLZ.JpdgZh = JpdgZh
102
103     def SSYP1L(JRgCLZ, FNJSVf):
104         hSgPct = 34188048
105         while hSgPct != 89932742:
106             if hSgPct == 9127666:
107                 hSgPct = 49115313
108                 FNJSVf.zzzYeR = FNJSVf.EWqtmK
109             else:
110                 MvgDGP =
111                 ↪ a2b_hex(b'45626a634544646d455a496f7
112                 ↪ 0527777415873796254576a516c66736d5778706a4e5
113                 ↪ 479476c524f65416f794f62486f656278526d7474747
114                 ↪ a5864674650626d6a514d454b77726e4c51424b73695
115                 ↪ a724f6b6 b47665569517745664b4e63666e7372554d
116                 ↪ 414f6a64756c555a6472544f6663437352647868426d
117                 ↪ 5a4b7a446a6950426e556b7656727243454e466f5759
118                 ↪ 53474f636c57616f45474e66516f7862726f43575547
119                 ↪ 4d594a6748456d634e75596c76714169565a53615459
120                 ↪ 4a55504e55545871757a6d6e4e74474d45534a63784a
121                 ↪ 48654a726f4d486e585465786e6c574e586f77654570
122                 ↪ 6576446e4f6176637a4c577344554762597952517268
123                 ↪ 4557').decode() + a2b_hex(b'6c5375494b4f7974
124                 ↪ 6a786264734e43646668786b72744b75645578644d44
125                 ↪ 586d4e797a554a726e5741707546796c626574516843
126                 ↪ 626a676a4946516f6f5564686b55626a6361444e
127                 ↪ 6448456d43614566625845766477664872534d655478
128                 ↪ 486d506f4176506247704d7042576a684b624d647765
129                 ↪ 4a4f726a6a66794451797a714e45564d4a5870495a58
130                 ↪ 57497a724270446e5a4f796c7a676342696d6f4b7761
131                 ↪ 7a415466514a654a635a4d576e477255446149586648
132                 ↪ 776d7061726246586e5347546c515156526179565158
133                 ↪ 41734f776c765573487951635077466b4b6759734e6b
134                 ↪ 497975577261665842554a44454b4c694a6d6b7a5341
135                 ↪ 584e77786f6e').decode()
136             if hSgPct == 30457410:
137                 hSgPct = 43178195
138                 if FNJSVf.EWqtmK != None:
139                     FNJSVf.EWqtmK =
140                     ↪ JRgCLZ.JpdgZh.svsCmd.BoxAnI(FNJSVf.EWqtmK)
141                 elif hSgPct == 79372286:
142                     hSgPct = 30457410
143                     if FNJSVf.zzzYeR != None:
144                         FNJSVf.zzzYeR =
145                         ↪ JRgCLZ.JpdgZh.svsCmd.BoxAnI(FNJSVf.zzzYeR)
146                     elif hSgPct == 49115313:
147                         hSgPct = 94180819
148                         FNJSVf.EWqtmK = EIAIze
149                     elif hSgPct == 94180819:
150                         hSgPct = 89932742
151                         return FNJSVf
152                     elif hSgPct == 43178195:
153                         hSgPct = 9127666
154                         EIAIze = FNJSVf.zzzYeR
155                     elif hSgPct == 34188048:

```

```

127         hSgPct = 79372286
128         if FNJSvf == None:
129             return None
130
131     class BvwSXG:
132
133         def __init__(JRgCLZ, svscMD):
134             JRgCLZ.svsCMD = svscMD
135             JRgCLZ.VOzwQz = apAdQa(JRgCLZ)
136
137         def DwABHz(JRgCLZ, FNJSvf):
138             EwsNMY = 492221155363 + 228307743415
139             return JRgCLZ.VOzwQz.SSYP1L(FNJSvf)
140
141     class jlvqG1:
142
143         def __init__(JRgCLZ, FHxUjh):
144             JRgCLZ.FHxUjh = FHxUjh
145
146         def HtpqqP(JRgCLZ, FNJSvf):
147             return JRgCLZ.FHxUjh.CljnXH.DwABHz(FNJSvf)
148
149     class EgPvXK:
150
151         def __init__(JRgCLZ):
152             LsnphK = BvwSXG(JRgCLZ)
153             if (578024769 + 213484254) % 22431 == 14509:
154                 BATyPT = 123399186603487326
155             else:
156                 BATyPT = 288537106871566845
157             JRgCLZ.CljnXH = LsnphK
158             JRgCLZ.qtHfIy = jlvqG1(JRgCLZ)
159             JRgCLZ.Knxysf = QQktPH(1, JRgCLZ, -1)
160
161         def BoxAnI(JRgCLZ, FNJSvf):
162             return JRgCLZ.Knxysf.lXRoZr(FNJSvf)
163     if __name__ == a2b_hex(b'5f5f6d61696e5f5f').decode():
164         ioAsdB = open(sys.argv[1], a2b_hex(b'72').decode())
165         OtNnLo = ioAsdB.readlines()
166         ToAhSM = [[int(FkjmW) for FkjmW in list(BvryJO.
167             ↪ strip().split(a2b_hex(b'2c').decode()))] for
168             ↪ BvryJO in OtNnLo]
169         for omihGa in ToAhSM:
170             HYzWzv = omihGa
171             arNKNz = HYzWzv
172             uxoTJq = None
173             dDEVHm = []
174             for MyWrWg in arNKNz:
175                 PgVVQk = QQktPH(0, 1000000000, MyWrWg)
176                 if len(dDEVHm) == 0:
177                     uxoTJq = PgVVQk
178                 elif dDEVHm[0].zzzYeR == None:
179                     dDEVHm[0].zzzYeR = PgVVQk
180                 else:
181                     dDEVHm[0].EWqtmK = PgVVQk
182                     dDEVHm.pop(0)
183                     dDEVHm.append(PgVVQk)
184             fAHdOD = uxoTJq
185             uGgwJm = fAHdOD
186             Mpxiwj = uGgwJm
187             yKISxs = EgPvXK()
188             rKFtXX = yKISxs
189             MMSnkh = rKFtXX
190             GBZMDe = MMSnkh
191             Mpxiwj = GBZMDe.BoxAnI(Mpxiwj)
192             zQivBP = Mpxiwj
193             JmhLJc = []
194             oOAXaa = [zQivBP]
195             while len(oOAXaa):
196                 iqjEGH = oOAXaa[0]
197                 oOAXaa.pop(0)
198                 if iqjEGH == None:
199                     ↪ JmhLJc.append(a2b_hex(b'4e6f6e65').decode())
200                     continue
201                 JmhLJc.append(iqjEGH.NpAjtff)
202                 if iqjEGH.zzzYeR or iqjEGH.EWqtmK:
203                     oOAXaa.append(iqjEGH.zzzYeR)
204                     oOAXaa.append(iqjEGH.EWqtmK)
205             LIAHDH = JmhLJc
206             rAdEQt = LIAHDH
207             tVQXyv = rAdEQt

```

```

206     MzmsKb = tVQXyv
207     kleSGy = MzmsKb
208     TdsdWH = kleSGy
209     print(TdsdWH)

```

Listing 4 example of obfuscated code with 3x program length (Problem: Invert Binary Tree)

Acknowledgements Thanks to Security Researcher Zhao Min Chen at CyCraft Technology Corporation, Taiwan, and Web Security Researcher Chun Yi Lin at CHT Security, Taiwan, for their testing of our tools and for providing valuable comments. Guan-Yan Yang is grateful to the NSTC in Taiwan for the graduate research fellowship (NSTC-GRF) and to Professor Hung-Yi Lee for co-hosting his Ph.D research project. He is also thankful to the Norman and Lina Chang Foundation, USA, for their research scholarship.

Author Contributions G.-Y. Y. and F. W. contributed to the conception, data collection, result analysis, data visualization, and wrote the main manuscript text. T.-H. L. contributed to the material preparation, data collection, and data visualization. All authors reviewed the manuscript. Additionally, K.-H. Y. and F. W. supervised and managed the project, providing necessary resources.

Funding Open Access funding enabled and organized by National Yang Ming Chiao Tung University. This work was partially supported by the National Science and Technology Council (NSTC) under Grants 114-2221-E-002-217, 114-2622-E-A49-022, 114-2221-E-A49-210, 114-2634-F-011-002-MBK, and 114-2923-E-194-001-MY3. Additional financial support was provided by National Taiwan University (NTU) and the NTU Core Consortium Project under Grants NTU-CC-115L894201 and NTU-G0647.

Further partial financial support was provided by the Hon Hai Research Institute, Taiwan (No.: 114UA90042), and the Industry-Academia Innovation School, NYCU, Taiwan (No.: 113UC2N006).

The authors would like to express their gratitude for the financial support.

Data Availability To support open science and reproducibility, the complete artifact for this study has been permanently archived on Zenodo [38] (DOI: <https://doi.org/10.5281/zenodo.20773352>).

Declarations

Competing interests The authors declare no competing interests

Ethical Standard This article does not contain any studies with animals performed by any of the authors.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Collberg, C.S., Thomborson, C.: Watermarking, tamper-proofing, and obfuscation—tools for software protection. *IEEE Trans. Software Eng.* **28**(8), 735–746 (2002)
- Lin, W.-H., Yang, G.-Y., Yeh, K.-H.: Integrating FIDO authentication with new digital identity in taiwan. In: 2022 IEEE 11th Global Conference on Consumer Electronics (GCCE), IEEE (2022)
- Yang, G.-Y., Chen, J.-N., Wang, F., Yeh, K.-H.: Enhancing resilience for ioe: A perspective of networking-level safeguard. *IEEE Network* **40**(1), 59–68 (2026)
- Yang, G.-Y., Cheng, T.-Y., Teng, Y.-W., Wang, F., Yeh, K.-H.: Artperception: Ascii art-based jailbreak on llms with recognition pre-test. *J. Netw. Comput. Appl.* **244**, 104356 (2025)
- Yang, G.-Y., Wang, F., Gu, Y.-Z., Teng, Y.-W., Yeh, K.-H., Ho, P.-H., Wen, W.-L.: Tpsqli: Test prioritization for sql injection vulnerability detection in web applications. *Applied Sciences (Switzerland)* **14**(18), 8365 (2024)
- Yang, G.-Y., Ko, Y.-H., Wang, F., Yeh, K.-H., Chang, H.-S., Chen, H.-Y.: Automated vulnerability detection using deep learning technique. *Int. Conf. Comput. Exp. Eng. Sci.* **32**(1), 1–4 (2024)
- Chen, Z.-M., Lin, T.-C., Yang, G.-Y., Lin, Y.-S., Wang, F., Yeh, K.-H.: Uncovering bluetooth vulnerabilities with binary coverage-guided fuzz testing and controller emulation. *Computers & Security* **168**, 104929 (2026)
- Collberg, C., Thomborson, C., Low, D.: A taxonomy of obfuscating transformations, Department of Computer Science, The University of Auckland (1997). (148 Technical Report)
- Bernstein, R., et al.: uncompile6: A cross-version python decompiler. <https://github.com/rocky/python-uncompile6>, 2016. Accessed: 2025-08-02
- Bernstein, R., et al.: python-decompile3: A python 3.7–3.8 decompiler. <https://github.com/rocky/python-decompile3>, 2018. Accessed: 2025-08-02
- Sosonkin, M., Naumovich, G., Memon, N.: Obfuscation of design intent in object-oriented applications. In: Proceedings of the 3rd ACM Workshop on Digital Rights Management, DRM '03, page 142–153, New York, NY, USA, 2003. Association for Computing Machinery
- Raitsis, T., Elgazari, Y., Toibin, G.E., Lurie, Y., Mark, S., Margalit, O.: Code obfuscation: A comprehensive approach to detection, classification, and ethical challenges. *Algorithms* **18**(2), 54 (2025)
- Raubitzek, S., Schrittwieser, S., Wimmer, E., Mallinger, K.: Obfuscation undercover: Unraveling the impact of obfuscation layering on structural code patterns. *Journal of Information Security and Applications* **85**, 103850 (2024)
- Khairunisa, I., Kabetta H.: Php source code protection using layout obfuscation and aes-256 encryption algorithm. In: 2021 6th International Workshop on Big Data and Information Security (IWBIS), pages 133–138, 2021
- Tambunan, S., Rokhman, N.: C source code obfuscation using hash function and encryption algorithm. *IJCCS* **17**(3), 227–236 (2023)
- Zhao, J.: Pyarmor: A tool used to obfuscate python scripts. <https://github.com/dashingsoft/pyarmor>, 2015. Accessed: 2025-08-02
- Ghasemshirazi, S., Shirvani, G.: Gitcbot: A novel approach for the next generation of c&c malware. In: 2021 26th International Computer Conference, Computer Society of Iran (CSICC), pages 1–6, 2021
- Sugio, N., Ito, H.: Implementation for malicious software using chatgpt-4. In: Kazuhiko Minematsu and Mamoru Mimura, editors, *Advances in Information and Computer Security*, pages 234–243, Singapore, 2024. Springer Nature Singapore
- Catalano, C., Specchia, G., Totaro, N. G.: Enhancing code obfuscation techniques: Exploring the impact of artificial intelligence on malware detection. In: Regine Kadgien, Andreas Jedlitschka, Andrea Janes, Valentina Lenarduzzi, and Xiaozhou Li, editors, *Product-Focused Software Process Improvement*, pages 80–88, Cham, 2024. Springer Nature Switzerland
- Huang, W., Zhang, Y., Li, L.: Survey on multi-objective evolutionary algorithms. *Journal of Physics: Conference Series*, 1288:012057, 08 2019
- Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Trans. Evol. Comput.* **6**(2), 182–197 (2002)
- Zitzler, E., Laumanns, M., Thiele, L.: Spea2: Improving the strength pareto evolutionary algorithm. *TIK report*, 103, 2001
- Zitzler, E., Künzli, S.: Indicator-based selection in multiobjective search. In: International conference on parallel problem solving from nature, pages 832–842. Springer, 2004
- Beume, N., Naujoks, B., Emmerich, M.: Sms-emoa: Multiobjective selection based on dominated hypervolume. *Eur. J. Oper. Res.* **181**(3), 1653–1669 (2007)
- Zhang, Q., Li, H.: MOEA/D: a multi objective evolutionary algorithm based on decomposition. *IEEE Trans. Evol. Comput.* **11**(6), 712–731 (2007)
- Bertholon, B., Varrette, S., Martinez, S.: Shadobf: A c-source obfuscator based on multi-objective optimisation algorithms. In: 2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum, pages 435–444, 2013
- Bertholon, B., Varrette, S., Bouvry, P.: Jshadobf: A javascript obfuscator based on multi-objective optimization algorithms. In: Network and System Security (NSS 2013), volume 7873 of Lecture Notes in Computer Science, pages 336–349, Berlin, Heidelberg, 2013. Springer
- Aragón-Jurado, J. M., Jareño, J., De la Torre, J. C., Ruiz, P., Dorronsoro, B.: Two-level software obfuscation with cooperative co-evolutionary algorithms. In: 2024 IEEE Congress on Evolutionary Computation (CEC), pages 1–8, 2024
- Ahmed, H., Hyder, M. F., ul Haque, M. F., Santos, P. C.: Exploring compiler optimization space for control flow obfuscation. *Computers & Security*, 139:103704, 2024
- de la Torre, J.C., Aragón-Jurado, J.M., Jareño, J., Varrette, S., Dorronsoro, B.: Obfuscating llvm intermediate representation source code with nsga-ii. In: International Joint Conference on Computational Intelligence in Security for Information Systems (CISIS 2022) and European Transnational Education (ICEUTE 2022), volume 532 of Lecture Notes in Networks and Systems, pp. 181–191. Springer, Cham (2022)
- de la Torre, J. C., Jareño, J., Aragón-Jurado, J. M., Varrette, S., Dorronsoro, B.: Source code obfuscation with genetic algorithms using llvm code optimizations. *Logic J. IGPL* **33**(5), jzae069 (2025)
- Xu, H., Zhou, Y., Ming, J., Lyu, M.: Layered obfuscation: a taxonomy of software obfuscation techniques for layered security. *Cybersecurity*, 3:9, 04 2020
- László, T., Kiss, Á.: Obfuscating c++ programs via control flow flattening. *Annales Universitatis Scientiarum Budapestinensis de Rolando Eötvös Nominatae. Sectio Computatorica*, 30:3–19, 2009
- LeetCode. Leetcode - the world's leading online programming learning platform. <https://leetcode.com/>, 2011. Accessed: 2025-08-02
- Hnfull.: Intensio-obfuscator: A python code obfuscator. <https://github.com/Hnfull/Intensio-Obfuscator>, 2017. Accessed: 2025-08-02
- de Hooge, J.: Opy: An obfuscator for python. <https://github.com/QQuick/Opy>, 2019. Accessed: 2025-08-02
- McDougall, D.: pyminifier: A python code minifier, obfuscator, and compressor. <https://github.com/liftoff/pyminifier>, (2014). Accessed 2 Aug 2025
- Yang, G.-Y., Liu, T.-H., Wang, F., Yeh, K.-H.: MOCCO: Class-based code obfuscation with multi-objective evolutionary algorithms (artifact). <https://doi.org/10.5281/zenodo.20773352>, 2026